

从联想记忆的视角理解 Transformer

钟澍^{*,†}, 徐明宇^{*}, 敖腾龙^{*}, 时光[†]

字节跳动 Seed

* 同等贡献, † 通讯作者

Abstract

本文中，我们分享了通过联想记忆——一个受人类认知启发的经典心理学概念——的视角来理解 Transformer 架构的思考与见解。我们从联想记忆的基础（例如简单的线性注意力）出发，然后深入探讨两个维度：

Memory Capacity: Transformer 究竟能记住多少，效果如何？我们引入检索信噪比来衡量这一点，并使用核视角从数学上揭示 Softmax 注意力如此有效的原因。我们还展示了前馈网络如何可被视为一种联想记忆，从而引出对其设计及潜在改进的见解。

Memory Update: 这些记忆如何学习和演化？我们提出了一个统一框架，用于理解不同 Transformer 变体（如 DeltaNet 和 Softmax 注意力）如何更新其“知识库”。这引导我们探讨两个引人深思的问题：1. *Are Transformers fundamentally limited in what they can express, and can we break these barriers?* 2. *If a Transformer had infinite context, would it become infinitely intelligent?*

我们希望揭开 Transformer 架构的神秘面纱，为现有设计提供更清晰的理解。这一探索旨在提供新的见解，并激发 Transformer 创新的新途径。

Date: 2026 年 1 月 28 日

Correspondence: 钟澍，邮箱：zhongshu@bytedance.com；时光，邮箱：shiguang@bytedance.com

1 Introduction

Transformer 模型 [42] 已成为现代人工智能（AI）的基础架构，在自然语言处理 [4]、计算生物学 [16]、计算机视觉 [7] 以及多模态任务 [29, 30] 等多个领域展现出卓越的通用性和突破性的性能。尽管其成功显而易见，但 Transformer 的内部机制依然复杂且未被完全理解，这为理论解释和基于原理的改进带来了重大挑战。

在我们探索 Transformer 架构奥秘的旅程中，我们发现源自人类认知的联想记忆概念，提供了一个独特且富有洞察力的直观视角。从心理学角度而言，联想记忆描述了大脑形成并回忆看似无关实体之间联系的能力 $\langle\langle \text{LEX}_C MD_6 \rangle\rangle$

有趣的是，我们发现 Transformer 架构——包括其核心组件自注意力机制和前馈网络（FFNs）——可以被优雅地理解为联想记忆的实现。更具体地说，注意力机制动态地形成短期的、上下文相关的关联，而 FFNs 则编码了在训练过程中提炼出的持久的、长期的联想记忆。这一统一的视角使得对 Transformer 行为的理解更为深入和结构化，并为架构创新提供了实用的见解。

我们的分析始于联想记忆的经典累积外积模型，这本质上是线性注意力机制的最简单表示。从这个起点

- *If a Transformer's context length grows infinitely, will the model itself become infinitely intelligent?*

关于第一个问题，我们讨论了模型的表达能力，因为当以电路复杂度衡量时，不同的记忆更新策略会显著影响模型的表达能力。例如，采用 delta 规则更新策略的 DeltaNet，相较于使用 Softmax 注意力的 Transformer，展现出更优的表达能力 [13, 24, 38]。基于这一洞见，我们引入了一种新颖的联想记忆模型，它结合了 Softmax 注意力的高检索精度与 DeltaNet 的 delta 规则更新机制，从而实现了改进的记忆管理与稳定性。通过理论证明，我们展示了所提出模型的表达能力超越了标准 Transformer，揭示了其可观的潜力。

对于第二个问题，一种常见的直觉认为：“支持更长上下文的 Transformer 能够解锁更大的智能。”例如，推理语言模型在获得更多思考时间后，能够解决日益复杂的问题。这引出了一个问题：从理论上讲，构建一个支持无限上下文以实现更大潜在智能的 Transformer 是否可行？上下文学习实际上对应于联想记忆的持续更新。从联想记忆更新的视角，我们发现理论上随着上下文增加，记忆可能逐渐收敛，导致上下文学习性能下降。这表明构建支持无限上下文的 Transformer 存在潜在的理论挑战。此外，我们还深入分析了几个可能阻止记忆收敛的缓解因素。

尽管已有若干研究探索了 Transformer 架构的联想记忆解释 [3, 31, 45]，我们希望我们的探索能提供新的见解、更清晰的理论理解，并为未来的 Transformer 创新带来启发。在结论部分，我们的探索最终导向一个前瞻性的讨论，受这些联想记忆原理的启发，我们提出了几个关于 AI 架构未来发展方向的基本问题。¹

2 Associative Memory

What is associative memory? 根据维基百科²，联想记忆是指学习和记忆不同实体之间关系的能力，即使这些实体本身并不直接相关。例如，一个访问过巴黎并见过埃菲尔铁塔的人，很可能在大脑中形成巴黎与埃菲尔铁塔之间的关联。因此，当未来提及巴黎时，他很可能会回忆起埃菲尔铁塔。从学术角度描述这一过程，巴黎作为一个key，触发了对存储在联想记忆中相应value的回忆，在此例中即埃菲尔铁塔。这种能力支撑着人类的认知功能，使我们能够从单一想法或感知（键）中高效检索相关信息（值）。

How to formalize the associative memory? 让我们定义一个矩阵 $\mathbf{S}$ 来表示联想记忆。 $\mathbf{S}$ 持续更新并随时间变化。例如，通过添加下标 t 得到 $\mathbf{S}_t$ ，我们表示在时间 t 的联想记忆。我们可以将每个时刻要存储的“信息”定义为一个键-值向量对 $(\mathbf{k}, \mathbf{v})$ ，其中 $\mathbf{k} \in \mathbb{R}^{d_k}, \mathbf{v} \in \mathbb{R}^{d_v}, \mathbf{S} \in \mathbb{R}^{d_v \times d_k}$ 。

键-值对如何被 $\mathbf{S}$ 具体记忆？一种经典的建模方法是直接累加键-值对的外积矩阵：

$$\mathbf{S}_t = \sum_{i=1}^t \mathbf{v}_i \mathbf{k}_i^\top. \quad (1)$$

那么如何根据键 $\mathbf{k}$ 从记忆 $\mathbf{S}$ 中检索存储的值 $\mathbf{v}$ ？我们可以定义一个由 $\mathbf{S}$ 参数化的函数 $f_{\mathbf{S}}: \mathbb{R}^{d_k} \rightarrow \mathbb{R}^{d_v}$ ，使得对于任何已存储的 $(\mathbf{k}_i, \mathbf{v}_i)$ ， $f_{\mathbf{S}}(\mathbf{k}_i)$ 尽可能接近 $\mathbf{v}_i$ 。 $f_{\mathbf{S}}$ ，也称为联想映射，本质上模拟了记忆的回忆过程。该回忆过程通常具有容错性 [15, 19]。即使 $\mathbf{k}_i$ 受到一定扰动，定义为查询 $\mathbf{q}_i \approx \mathbf{k}_i$ ， $f_{\mathbf{S}}(\mathbf{q}_i)$ 仍然可以回忆起一个相似的 $\mathbf{v}_i$ 。一个经典的 $f_{\mathbf{S}}$ 是简单的线性变换，具体为 $f_{\mathbf{S}}(\mathbf{q}) = \mathbf{S}\mathbf{q}$ 。

¹请将本文视为一篇博客风格的个人思考分享，而非正式的研究论文，旨在激发如同团队协作会议中的讨论。

²[https://en.wikipedia.org/wiki/Associative_memory_\(psychology\)](https://en.wikipedia.org/wiki/Associative_memory_(psychology))

总之，我们发现联想记忆模型由两个组成部分决定： $\mathbf{S}$ 的更新机制和联想映射 $f_{\mathbf{S}}$ 。

Transformer can actually be interpreted as associative memory. 以 Transformer 的核心组件——注意力层为例，其在时间 t 的输出 $\mathbf{o}_t$ 可计算为：

$$\mathbf{o}_t = \sum_{i=1}^t \frac{\exp(\mathbf{k}_i^\top \mathbf{q}_t / \sqrt{d_k})}{\sum_{j=1}^t \exp(\mathbf{k}_j^\top \mathbf{q}_t / \sqrt{d_k})} \mathbf{v}_i. \quad (2)$$

定义对应于 $\exp$ 核函数的特征映射为 $\phi(\cdot)$ ，使得 $\exp(\mathbf{k}^\top \mathbf{q} / \sqrt{d_k}) = \phi(\mathbf{k})^\top \phi(\mathbf{q})$ 。为简化起见，我们忽略归一化项。 $\mathbf{o}_t$ 的计算可以重写为：

$$\begin{aligned} \mathbf{o}_t &= \sum_{i=1}^t \phi(\mathbf{k}_i)^\top \phi(\mathbf{q}_t) \mathbf{v}_i \\ &= \underbrace{\sum_{i=1}^t \mathbf{v}_i \phi(\mathbf{k}_i)^\top}_{\mathbf{S}_t} \phi(\mathbf{q}_t). \end{aligned} \quad (3)$$

因此，我们发现注意力层是联想记忆的一种形式，其中 $\mathbf{S}_t = \sum_{i=1}^t \mathbf{v}_i \phi(\mathbf{k}_i)^\top$ 和联想映射 $f_{\mathbf{S}}(\mathbf{q}) = \mathbf{S} \phi(\mathbf{q})$ 。事实上，Transformer 的另一个核心组件——前馈网络（FFN）——也可以被视为联想记忆的一种形式（参见第 2.1.1 节中关于 ReLU 核的讨论）。

尽管注意力层和前馈网络（FFN）都维护着可归类为联想记忆的记忆，但它们在存储信息的寿命方面存在差异。具体而言，注意力层维护的是一种以联想方式组织的短期上下文记忆。在推理过程中，这种被称为键值（KV）缓存的记忆在推理完成后即被丢弃。相比之下，FFN 维护的是一种持久、长期的联想记忆。这种记忆在训练期间通过梯度下降进行压缩，并编码了与训练数据集相关的知识。通常，它在训练结束后保持不变。

由于 Transformer 属于联想记忆的一种类型，我们可以从联想记忆的角度来理解 Transformer 及其各种变体。作为一种记忆，很自然地可以从两个维度考察其特性：记忆容量（第 2.1 节）和记忆更新（第 2.2 节）。为了使讨论具有普遍性，即涵盖尽可能多的模型设计，针对每个维度，我们将从对应于公式 1 的基本模型出发，逐步扩展并分析不同的影响因素。

2.1 Memory Capacity

具体而言，考虑 $\mathbf{q}_t = \mathbf{k}_i$ ，检索任意 $\mathbf{v}_i$ 的操作如下：

$$\mathbf{o}_t = \mathbf{S}_t \mathbf{k}_i = \underbrace{\mathbf{v}_i (\mathbf{k}_i^\top \mathbf{k}_i)}_{\text{signal term}} + \underbrace{\sum_{j \neq i} \mathbf{v}_j (\mathbf{k}_j^\top \mathbf{k}_i)}_{\text{noise term}}. \quad (4)$$

为实现精确检索，要求向量 $\{\mathbf{k}_1, \dots, \mathbf{k}_t\}$ 是标准正交的，这意味着在公式 4 中，对于 $j \neq i$ ，有 $\mathbf{k}_i^\top \mathbf{k}_i = 1$ 和 $\mathbf{k}_j^\top \mathbf{k}_i = 0$ 。

关联记忆 $\mathbf{S}_t$ 的容量是多少？也就是说，在保证精确回忆的条件下，它最多能存储多少对键值对？直观上，由向量 $\mathbf{k}_i$ 张成的线性空间仅有 d_k 个正交基。如果存储的键值对数量 $n \gg d_k$ ，则公式 4 中的“噪

声”项 $\sum_{j \neq i} \mathbf{v}_j(\mathbf{k}_j^\top \mathbf{k}_i)$ 将变为非零，从而导致检索误差增加。

为了定量分析模糊关联记忆检索的性能，我们定义了一个类似于信噪比（SNR）的度量指标。具体来说，考虑一般的检索操作：

$$\mathbf{S}_t \mathbf{k}_i = c \mathbf{v}_i + \mathbf{r}, \quad (5)$$

其中第一项代表信号，由因子 c 缩放，第二项代表噪声 $\mathbf{r}$ 。因此，逆信噪比定义为：

$$\text{SNR}^{-1} = \mathbb{E}_{\mathbf{v}_j, \mathbf{k}_j} \left[\frac{\|\mathbf{r}\|^2}{c^2 \|\mathbf{v}_i\|^2} \right]. \quad (6)$$

该指标直观地衡量了噪声相对于信号的比例。值越大表明噪声成分越高，导致检索精度越低。我们考虑一种场景，其中键向量 $\mathbf{k}_j \in \mathbb{R}^{d_k}$ 和值向量 $\mathbf{v}_j \in \mathbb{R}^{d_v}$ 是独立同分布（i.i.d.）的标准高斯向量。在此假设下，我们可以定量计算逆信噪比，从而提供一个衡量检索精度下降的显式度量。

对于公式 4 中的线性形式，信号项为 $c = \mathbf{k}_i^\top \mathbf{k}_i$ ，噪声由 $\mathbf{r} = \sum_{j \neq i} \mathbf{v}_j(\mathbf{k}_j^\top \mathbf{k}_i)$ 给出。如附录 A 所示，这导出了以下近似表达式：

$$\boxed{\text{SNR}_{\text{Linear}}^{-1} \approx \frac{N}{d_k}}. \quad (7)$$

这一结果直观地表明，噪声随着存储的键值对数量 N 线性增加³，并随着键向量维度 d_k 的增加而反比减小。实际上，这意味着对于线性形式的可靠关联回忆，存储的对数应大约在键向量的维度数量级或更少： $d_k \gtrsim N$ 。

超过此阈值，由于噪声的累积，检索精度会迅速恶化，从而严重限制了关联记忆的有效容量。这也解释了为什么线性 Transformer 在精确的长上下文检索任务中常常表现不佳 [20, 28]。

接下来，我们介绍两种提高关联记忆容量的策略：核映射和稀疏性。这些技术分别广泛应用于注意力机制和前馈层。虽然传统上被视为独立的设计选择，但我们表明它们并非互斥，实际上具有显著的结构相似性。

2.1.1 Kernel Mapping

由于记忆容量受限于 d_k ，我们可以应用核函数将向量映射到更高维的空间，使得查询-键匹配能够在这个扩展的空间中进行。直观上，在更高维的空间中，向量之间变得更加可区分，从而更容易被分离，这减少了检索误差，并使记忆能够存储更多的值 $\mathbf{v}$ 。这一洞见与 Johnson-Lindenstrauss 引理和叠加理论密切相关 [9, 14]。

通过引入核函数 $\kappa(\mathbf{x}, \mathbf{y}) = \phi(\mathbf{x})^\top \phi(\mathbf{y})$ ，映射过程变为 $f_{\mathbf{S}_t}(\mathbf{k}_t) = \mathbf{S}_t \phi(\mathbf{k}_t)$ 。相应地，联想记忆和回忆操

³我们使用 N 而非先前的变量 t ，以强调存储键值对的总 *number*。

作如下：

$$\mathbf{S}_t = \sum_{i=1}^t \mathbf{v}_i \phi(\mathbf{k}_i)^\top, \quad (8)$$

$$\mathbf{o}_t = \mathbf{S}_t \phi(\mathbf{q}_t) = \sum_{i=1}^t \mathbf{v}_i (\phi(\mathbf{k}_i)^\top \phi(\mathbf{q}_t)) = \sum_{i=1}^t \mathbf{v}_i \kappa(\mathbf{k}_i, \mathbf{q}_t). \quad (9)$$

我们现在引入核化设置下的信噪比分析。遵循附录A中的推导，我们得到：

$$\text{SNR}_\kappa^{-1} = N \frac{\mathbb{E}_{k_j} [\kappa^2(\mathbf{k}_j, \mathbf{k}_i)]}{\kappa^2(\mathbf{k}_i, \mathbf{k}_i)}. \quad (10)$$

在核函数为线性的特殊情况下，即 $\kappa(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{y}$ ，核化公式恢复了公式7中原始线性情况下的逆信噪比。接下来，我们分析两个在联想记忆中特别重要的核函数：指数核和 ReLU 核，因为它们与现代 Transformer 架构中常用的注意力机制和前馈网络紧密相关。

Exp Kernel. 我们将指数核定义为 $\kappa_{\text{exp}}(\mathbf{x}, \mathbf{y}) = \exp(\frac{\mathbf{x}^\top \mathbf{y}}{\tau})$ ，其中 $\tau > 1$ 是一个温度参数。通常， τ 被设置为 $\sqrt{d_k}$ 以确保适当的缩放，因为点积 $\mathbf{x}^\top \mathbf{y}$ 的方差为 d_k （假设各分量独立、均值为零、方差为一）。通过这一选择，核公式直接对应于标准的 Softmax 注意力机制⁴： $\sum_{i=1}^t \mathbf{v}_i \exp(\mathbf{k}_i, \mathbf{q}_t)$ 。事实上，指数核允许一个显式的特征映射到无限维的希尔伯特空间，这可以表示为一个无穷幂级数⁵：

$$\phi(\cdot) = \begin{bmatrix} 1 \\ (\cdot) \\ \frac{(\cdot)^{\otimes 2}}{\sqrt{2!}} \\ \vdots \\ \frac{(\cdot)^{\otimes \infty}}{\sqrt{\infty!}} \end{bmatrix}, \quad (11)$$

其中 $(\cdot)^{\otimes m}$ 表示向量与自身的 m 重克罗内克积。为清晰起见，我们在此省略了 τ 。具体来说，我们有，

$$\mathbf{S}_t \phi(\mathbf{q}_t) = \sum_{i=1}^t (\phi(\mathbf{q}_j)^\top \phi(\mathbf{k}_i)) \mathbf{v}_i = \sum_{i=1}^t \sum_{m=0}^{\infty} \frac{(\mathbf{q}_t^{\otimes m})^\top (\mathbf{k}_i^{\otimes m})}{m!} \mathbf{v}_i = \sum_{i=1}^t \sum_{m=0}^{\infty} \frac{(\mathbf{q}_t^\top \mathbf{k}_i)^m}{m!} \mathbf{v}_i = \sum_{i=1}^t \exp(\mathbf{q}_t^\top \mathbf{k}_i) \mathbf{v}_i. \quad (12)$$

在同样假设键和值均为独立同分布的标准高斯向量的前提下，如附录 A 所推导，指数核的逆信噪比可近似为：

$$\text{SNR}_{\text{exp}}^{-1} \approx \frac{N}{\exp(\frac{2(\tau-1)}{\tau^2} d_k)}. \quad (13)$$

⁴注意，归一化项被省略，因为它不影响逆信噪比的计算。

⁵实际上，由于精度有限，我们的实验发现该展开在实践中大约在 7 到 8 阶处截断，而非无限继续。

这一结果表明，指数核显著降低了精确检索所需的特征维度 d_k ：从线性情况（式 7）下的 $\mathcal{O}(N)$ 降至使用指数核且当 $\tau = \sqrt{d_k}$ 时的 $\mathcal{O}(\log^2 N)$ 。

这一理论发现可以得到若干实验观察的支持：

1. *Dimension Scaling in LLMs*: 在实际使用 Softmax Attention 的 LLM 训练中，随着上下文长度 N 的增长，为保持检索精度，增加维度 d 变得必要，这与我们推导出的信噪比关系 [46] 直接吻合。
2. *Temperature Effects*: 降低温度参数 τ ——同时保持其大于 1——能提高检索精度，这与 [27, 44] 的研究结果一致 $\tilde{a}$
3. *Multi-head Efficiency Trade-off*: 我们假设，Softmax Attention 卓越的检索能力使其能够通过多头架构以维度换取更强的表达能力，而线性注意力则更多受益于更高的维度。我们在第 3.1 节的初步实验支持这一结论。
4. **Linear Attention**. 线性注意力的优化目标是无界的，这可能导致 $\mathbf{S}\mathbf{k}$ 的范数爆炸，从而引发数值不稳定。门控线性注意力 [47] 引入了记忆衰减项 $\text{diag}(\boldsymbol{\lambda})$ ，这相当于在优化目标中对 $\mathbf{S}$ 的 Frobenius 范数添加了一个正则化项。这以记忆遗忘为代价缓解了范数爆炸问题。
5. **DeltaNet**. DeltaNet [33, 48] 将内积目标替换为 L2 回归目标，从而固有地对 $\mathbf{S}\mathbf{k}$ 施加了范数正则化。在循环形式中，这相当于引入了一个新的衰减项 $\mathbf{I} - \mathbf{k}\mathbf{k}^\top$ 。这种衰减的效果是在记忆写入前，检索并衰减与相似键相关联的历史值，从而防止更新后的范数爆炸。此外，可以将一阶动量纳入键值对更新中以调整记忆更新动态。Behrouz et al. [2] 发现，纳入动量对模型性能有积极影响。⁶
6. **Softmax Attention**. Softmax 注意力 [43] 通过引入指数核函数来提升记忆召回性能。如果没有 softmax 操作中的归一化项，优化目标仍然是无界的。在存在归一化项的情况下，对于足够长的上下文（大的 t ），根据附录 B.2 中的推导，目标函数包含对 $\mathbf{S}$ 的 Frobenius 范数的正则化，这缓解了数值不稳定性。此外，该目标函数近似地引入了因子 $\frac{1}{t}$ 。随着 t 增大，我们有 $\frac{1}{t} \rightarrow 0$ ，导致 $\mathcal{L}_t \rightarrow 0$ ，从而使得关于 $\mathbf{S}$ 的梯度逐渐消失。这可能会抑制记忆更新（详见第 2.2.2 节讨论）。与门控线性注意力类似，我们也可以通过将门控因子 $\text{diag}(\boldsymbol{\lambda}_t)$ 纳入参数矩阵 $\mathbf{A}_t$ 来为 softmax 注意力引入记忆衰减。Lin et al. [21] 发现这提升了 softmax 注意力的性能。
7. How to improve the accuracy of memory recall? 一种有效的方法是核技巧。例如，softmax 注意力采用了指数核函数。如第 2.1.1 节所讨论的，该方法通过在不增加空间复杂度的前提下执行额外的核计算，有效提升了记忆召回精度。
8. How to effectively manage memory update to avoid such as the explosive growth of the spectral norm of $\mathbf{S}$? 可以从数学上证明，形式为 $\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_i \mathbf{k}_i^\top$ 的记忆模型，无论是否使用核技巧，都不可避免地会经历 $\mathbf{S}$ 的谱范数随时间增长，从而导致数值不稳定。
9. **Kernel function**. 用于计算 $\mathbf{u}$ （公式 39）和用于记忆检索（公式 40）的核函数不必相同；可以独立采用不同的核函数 $\tilde{a}$
10. **Gate**. 门可以分别引入到计算 $\mathbf{u}$ （式 39）的两个部分中：项 $-\sum_{i=1}^{t-1} \phi(\mathbf{k}_i)^\top \phi(\mathbf{k}_t) \mathbf{u}_i$ 和向量 $\mathbf{v}_t$ 。

⁶[2] 中的模型与 DeltaNet 并不严格等价；因此，这一发现应仅作为参考。

11. Similarity calculation. 在计算 $\mathbf{u}$ (式 39) 时, 可以引入一个新的向量, 而非 $\mathbf{k}_t$, 来计算与 $\{\mathbf{k}_i\}_{i=1}^{t-1}$ 的相似度, 从而控制内存的写入与擦除。
12. NC^k . 电路规模在 n 上为多项式, 深度为 $\mathcal{O}(\log^k(n))$, 使用扇入有界 (通常限制为 2 或有限常数) 的与、或、非门。直观上, 这类似于雇佣大量工人并行执行任务, 但每个工人一次只能处理有限数量的任务 (小扇入)。可由 NC 电路解决的经典计算问题包括整数加法 ($k = 1$) 和矩阵乘法 ($k = 2$)。
13. AC^k . 电路规模在 n 上为多项式, 深度为 $\mathcal{O}(\log^k(n))$, 并使用扇入无限的与、或、非门。直观上, 这对应于雇佣更少但更强大的工人, 每个工人能够同时处理大量任务 (无限扇入)。这些电路可以在常数深度内执行简单的逻辑判断, 例如 “是否至少有一个输入等于 1?” 或 “是否所有输入都等于 1?”, 这需要无限扇入才能高效实现这些计算。
14. TC^k . 电路规模在 n 上为多项式, 深度为 $\mathcal{O}(\log^k(n))$, 并采用特殊的阈值门。阈值门计算等于 1 的输入数量, 如果该数量达到或超过某个阈值则输出 1, 否则输出 0。阈值门具有无限扇入。直观上, 这些电路更强大, 能够实现高效的 “多数表决”, 从而处理更复杂的计数和决策任务。可由 TC 电路解决的经典问题包括数值比较、计数和多数判定。
15. AC^0 vs. TC^0 : Can we compute the sum of n numbers in a single step? 尽管具有无限扇入, AC^0 只能执行简单的逻辑检查, 例如 “是否至少有一个输入等于 1?”, 但它无法直接计数有多少个输入等于 1。因此, AC^0 无法在单步内计算 n 个数的和。相比之下, TC^0 引入了阈值门, 能够在常数深度内对多个位 (0 或 1) 进行计数, 从而允许在单个 (常数深度的) 步骤中对多个数求和。
16. TC^0 vs. NC^1 : Can we perform arbitrary swaps within a sequence of arbitrary length? TC^0 的一个关键限制是其常数深度: 每个输出只能依赖于输入的一个常数大小邻域内的信息, 这使得任意长度的元素交换成为不可能。相反, NC^1 电路具有对数深度, 允许构建二叉树结构来高效地执行任意长度的交换。
17. Training phase. 首先, 我们计算序列 $\{\mathbf{u}\}_{i=1}^T$ 。观察式 39, 每个 $\mathbf{u}$ 的计算复杂度⁷为 $\mathcal{O}(Td)$ 。因此, 整个序列的计算复杂度总计为 $\mathcal{O}(T^2d)$ 。由于 $\mathbf{u}_t$ 依赖于历史 $\{\mathbf{u}\}_{i=1}^{t-1}$, 其顺序时间复杂度为 $\mathcal{O}(T)$ 。一旦计算出 $\{\mathbf{u}\}_{i=1}^T$, 式 40 中的操作将分别产生 $\mathcal{O}(T^2d)$ 和 $\mathcal{O}(1)$ 的计算复杂度和时间复杂度。因此, 训练期间的总计算复杂度和时间复杂度分别为 $\mathcal{O}(T^2d)$ 和 $\mathcal{O}(T)$ 。
18. Inference phase. 首先, 我们计算 $\mathbf{u}_t$ 并将其与 $\mathbf{k}_t$ 结合以更新 $\mathbf{S}_t$ 。相应的计算复杂度和时间复杂度分别为 $\mathcal{O}(Td)$ 和 $\mathcal{O}(1)$ 。接着, 采用 $\mathbf{q}_t$ 从 $\mathbf{S}_t$ 中检索最终输出 $\mathbf{o}_t$, 其计算复杂度和时间复杂度分别为 $\mathcal{O}(d^2)$ 和 $\mathcal{O}(1)$ 。因此, 推理阶段的整体计算复杂度和时间复杂度分别为 $\mathcal{O}(Td + d^2)$ 和 $\mathcal{O}(1)$ 。

ReLU Kernel. 我们证明, 标准的无偏置两层线性层加非线性 ReLU 激活的前馈网络 (FFN) 可被解释为由 ReLU 核定义的关联记忆⁸。考虑由 $\text{FFN}(\mathbf{x}) = \mathbf{W}_V^\top \text{ReLU}(\mathbf{W}_K \mathbf{x})$, 给出的 FFN 其中 $\mathbf{x} \in \mathbb{R}^d$ 是输入, $\mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{m \times d}$ 是权重矩阵, ReLU 函数逐元素应用: $\text{ReLU}(z) = \max\{0, z\}$ 。基于上述公式, 我

⁷ 此处计算复杂度定义为乘法运算的次数。

⁸ 其他类型的 FFN 激活函数可用类似方法分析; 此处我们选择最具代表性的类型。此外, 我们不单独分析 SiLU 激活, 因其行为与 ReLU 相似, 且不影响我们的结论。

们可以直接将 FFN 映射到一个关联记忆：

$$\begin{aligned}\text{FFN}(\mathbf{x}) &= \mathbf{W}_V^\top \text{ReLU}(\mathbf{W}_K \mathbf{x}) \\ &= \sum_{i=1}^m \underbrace{\mathbf{W}_{V_i}^\top}_{\mathbf{v}_i} \underbrace{\text{ReLU}(\mathbf{W}_{K_i} \mathbf{x})}_{\kappa(\cdot, \cdot)} \underbrace{\mathbf{x}}_{\mathbf{q}_i}.\end{aligned}\quad (14)$$

这与公式 9 一致，其中两个线性层 $\mathbf{W}_{U_i}, \mathbf{W}_{D_i}$ 中的行定义了 m 个记忆键 $\mathbf{k}_i$ 和值 $\mathbf{v}_i$ ，以及核函数 $\kappa_{\text{ReLU}}(\mathbf{x}, \mathbf{y}) = \text{ReLU}(\mathbf{x}^\top \mathbf{y})$ 。从这个视角，我们可以观察到 FFN 与注意力机制底层的记忆机制本质上是统一的。The key distinction is that Attention maintains context-dependent associative memories (e.g., the surname and given name of a new person in context), while FFNs encode persistent memories that reflect more general world knowledge (e.g., the name of a famous celebrity). 这种结构相似性表明它们的设计可以相互启发，并且这一点已被认识到 [42]。我们将在后续章节讨论这种潜在的整合。

根据附录 A 中给出的推导，我们将 ReLU 核的逆信噪比近似为：

$$\boxed{\text{SNR}_{\text{ReLU}}^{-1} \approx \frac{N}{2d_k}}. \quad (15)$$

与线性情况（公式 7）相比，ReLU 核通过将负相似度的查询-键对抑制为零来提升检索性能。然而，其精度仍显著低于注意力机制中使用的指数核。我们接下来将对此进行讨论。

2.1.2 Vignette 1: Rethinking Attention and FFN from the Perspective of Associative Memory

Table 1 注意力机制与前馈网络典型架构组件对比。

Component	Attention	FFN
Kernel	Exp	ReLU / SiLU
Normalization	Yes	None
Multihead	Yes	None
Sparsity	None	MoE
Gating	None	SwiGLU Gating

Transformer 中注意力机制与前馈网络之间的对称性引人入胜，因为——至少在原则上——为其中一个模块设计的任何方案都可以完全在另一个模块中实现。这种对称性迫使我们更深入地探究它们的差异，同时也激励我们去发现新的架构设计。

表 1 总结了两模块共享的几种最常见、最经典的设计。接下来我们将聚焦于它们在核心运算（核函数）层面的差异，然后简要讨论归一化、多头机制、稀疏性和门控机制。对这些主题更深入的探索留待未来工作；我们也鼓励读者思考为何存在这些差异，以及是否可以利用它们来改进双方。

Kernel. 如上所述，两模块所采用的不同核函数引出了一个有趣的问题：为什么前馈网络采用 ReLU 核函数，而不是像 Exp 核函数那样更精确的核函数？

我们提出以下假设：A kernel with lower retrieval precision encourages a more polysemantic key-value

memory: multiple unrelated facts can be stored under the same key space.这在可解释性研究中通常被称为叠加 [9, 14]。这增加了能够装入固定宽度关联记忆中的抽象、压缩知识的数量。相反，注意力层经常需要从当前上下文中精确定位一个特定的词元（例如归纳头 [25]），因此受益于高度精确、更具单义性的核函数。相比之下，前馈网络必须将整个训练语料库提炼为静态权重；因此，即使以召回时噪声更大为代价，能够在一个向量下叠加许多信息片段的能力也是有利的。

一项经验证据来自 Anthropic 用 Softmax 线性单元激活函数 [8] 替换 ReLU 的实验：

$$\kappa_{\text{SoLU}}(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^\top \mathbf{y}) \exp\left(\frac{\mathbf{x}^\top \mathbf{y}}{\tau}\right), \quad (16)$$

这可以看作是线性核与指数核的乘法混合。他们观察到隐藏表示变得显著 monosemantic，但模型在知识密集型基准测试 TriviaQA 上的准确性却下降了。

在附录 A 中，我们推导了 SoLU 核函数的逆信噪比：

$$\boxed{\text{SNR}_{\text{SoLU}}^{-1} \approx \frac{5N}{d_k \exp(2\sqrt{d_k})}}. \quad (17)$$

这表明 SoLU 提供了比 ReLU 更高的检索精度，这与观察到的其单义性行为一致。总之，这些发现支持了我们更广泛的假设：核函数的选择从根本上调节了神经架构中知识密度（存储的信息量）与检索精度（记忆访问的特异性）之间的权衡。我们将关于知识保持与上下文召回之间核函数选择的更全面探索留待未来工作。

Normalization. 与先前介绍的联想回忆公式（式 9）相比，标准的 Softmax 注意力包含一个额外的归一化因子：

$$\mathbf{S}_t \phi(\mathbf{q}_t) = \frac{\sum_{i=1}^t \mathbf{v}_i \exp(\mathbf{q}_t^\top \mathbf{k}_i)}{\sum_{i=1}^t \exp(\mathbf{q}_t^\top \mathbf{k}_i)}, \quad (18)$$

这是前馈网络所省略的。在将归一化融入前馈网络方面，与我们工作最密切相关的是 NormFormer [37]，它在非线性激活（核函数）之后立即引入了层归一化操作，从而提高了训练稳定性。由于许多线性注意力变体 [20, 40, 48] 已表明该因子可以被简单的每头归一化所取代，因此我们主要将归一化视为一种训练稳定机制，而非根本性的设计需求。

Multihead. 多头注意力将 $\mathbf{q}, \mathbf{k}, \mathbf{v}$ 切片为 n_h 个头，并使用输出矩阵 $\mathbf{W}_O \in \mathbb{R}^{d \times d_h n_h}$ 来组合每个头的结果：

$$[\mathbf{q}_{j,1}; \mathbf{q}_{j,2}; \dots; \mathbf{q}_{j,n_h}] = \mathbf{q}_j, \quad (19)$$

$$[\mathbf{k}_{i,1}; \mathbf{k}_{i,2}; \dots; \mathbf{k}_{i,n_h}] = \mathbf{k}_i, \quad (20)$$

$$[\mathbf{v}_{i,1}; \mathbf{v}_{i,2}; \dots; \mathbf{v}_{i,n_h}] = \mathbf{v}_i, \quad (21)$$

$$\mathbf{o}_{t,\cdot} = \sum_{i=1}^t \mathbf{v}_{i,\cdot} \kappa(\mathbf{q}_{t,\cdot}, \mathbf{k}_{i,\cdot}), \quad (22)$$

$$\mathbf{o}_t = \mathbf{W}_O[\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}]. \quad (23)$$

这种切片机制使得多头注意力能够为相同的查询-键对计算 n_h 种不同的相似性模式。正如我们之前对指数核的分析，当使用更强的核时，多头机制可以以检索精度换取表达能力，或者，增强叠加以提高知识容量。此外，从直观角度来看，多头注意力中的输出投影矩阵 $\mathbf{W}_O$ 可以减少式 4 中的噪声项⁹。在最初的 Transformer 论文中，作者尝试用多头注意力完全替代整个前馈网络层 [42]，观察到了一些性能提升。鉴于注意力机制在现代 GPU 上已得到显著优化 [5, 6, 34]，我们认为将多头机制引入前馈网络，即在式 14 中对 $\mathbf{x}, \mathbf{W}_{K_i}, \mathbf{W}_{V_i}$ 进行切片，值得重新考虑，我们将对此想法的探索留待未来工作。

Sparsity. 很容易看出，当键和值的数量变得很大时，与每个查询相关的键不可避免地是稀疏的。在前馈网络中融入稀疏性的最简单方法是通过专家混合机制 [11, 36]。考虑一个具有 E 个专家子网络的前馈网络，每个子网络由独立的权重矩阵 $\mathbf{W}_K^{(e)}, \mathbf{W}_V^{(e)} \in \mathbb{R}^{(m/E) \times d}$ 参数化。一个门控函数 $g_e(\mathbf{x})$ （通常是稀疏的）将输入路由到这些专家的一个小子集（通常是一个）：

$$\text{FFN}_{\text{MoE}}(\mathbf{x}) = \sum_{e=1}^E g_e(\mathbf{x}) \left(\sum_{i \in \text{Expert}(e)} \mathbf{W}_{V_i}^{(e)} \text{ReLU}(\mathbf{W}_{K_i}^{(e)} \mathbf{x}) \right). \quad (24)$$

为了在注意力机制中实现类似的稀疏性，可以将键值对划分为互不相交的专家子集，并基于查询实现稀疏路由：

$$\mathbf{o}_{t,\text{MoE}} = \sum_{e=1}^E g_e(\mathbf{q}_t, \{\mathbf{k}_i\}_{i \in \text{Expert}(e)}) \left(\sum_{i \in \text{Expert}(e)} \mathbf{v}_i \kappa(\mathbf{q}_t, \mathbf{k}_i) \right). \quad (25)$$

注意力机制与 FFN 在 MoE 机制中的核心区别在于，注意力中的键和值（ $\mathbf{k}$ 和 $\mathbf{v}$ ）是动态的，而 FFN 中的专家参数是静态的。因此，注意力中的门控函数 g_e 需要依赖于键向量集合 $\{\mathbf{k}_i\}_{i \in \text{Expert}(e)}$ 。例如，

⁹例如，在多查询注意力中，如果 $\mathbf{W}_O$ 学会计算共享键和值的 n_h 个头的简单平均值，那么与具有相同键/值维度的单头注意力相比，噪声项的方差会减少 n_h 倍。

MoBA 架构 [22] 将门控函数定义为（为清晰起见省略了归一化项）：

$$g_e(\mathbf{q}_t, \{\mathbf{k}_i\}_{i \in \text{Expert}(e)}) = \begin{cases} 1 & s_e(\mathbf{q}_t) \in \text{Topk}(\{s_{e'}(\mathbf{q}_t) \mid e' \in [E]\}, k) \\ 0 & \text{otherwise} \end{cases}, \quad (26)$$

$$s_e(\mathbf{q}_t) = \langle \mathbf{q}_t, \text{mean-pool}(\{\mathbf{k}_i\}_{i \in \text{Expert}(e)}) \rangle. \quad (27)$$

Gating. 在 FFN 中另一个普遍采用的架构选择是门控机制，以 SwiGLU 激活 [35] 为例。SwiGLU 门控在输入的两个线性变换之间引入了乘法交互，其定义为：

$$\text{FFN}_{\text{SwiGLU}}(\mathbf{x}) = \mathbf{W}_V^\top ((\mathbf{W}_G \mathbf{x}) \odot \text{Swish}(\mathbf{W}_U \mathbf{x})), \quad (28)$$

其中 $\mathbf{W}_K, \mathbf{W}_G, \mathbf{W}_V \in \mathbb{R}^{m \times d}$ 是参数矩阵， $\odot$ 表示逐元素乘法，Swish 函数由 $\text{Swish}(z) = z \cdot \text{Sigmoid}(z)$ 给出。

在联想记忆框架中表达，SwiGLU 可以重写为：

$$\text{FFN}_{\text{SwiGLU}}(\mathbf{x}) = \sum_{i=1}^m \underbrace{\mathbf{W}_{V_i}}_{\mathbf{v}_i} \underbrace{(\mathbf{W}_{G_i}^\top \mathbf{x})}_{g_i(\mathbf{x})} \underbrace{\text{Swish}(\mathbf{W}_{K_i}^\top \mathbf{x})}_{\kappa(\cdot, \cdot)} \underbrace{\mathbf{k}_i}_{\mathbf{k}_i} \underbrace{\mathbf{q}_t}_{\mathbf{q}_t}, \quad (29)$$

其中 $(\mathbf{W}_{G_i}^\top \mathbf{x})$ 是额外引入的门控项 $g_i(\mathbf{x})$ ，它可以放大或抑制查询对特定键的响应。直观上，这种门控机制提供了一种动态的、依赖于输入的调制。近期的线性变换器已经在注意力层中尝试了门控机制，例如门控线性注意力 [47]。其门控项定义为（省略归一化项）：

$$\mathbf{o}_{t, \text{gated}} = \sum_{i=1}^t \mathbf{v}_i g_i(\mathbf{x}_{i:t}) \kappa(\mathbf{q}_t, \mathbf{k}_i), \quad (30)$$

$$g_i(\mathbf{x}_{i:t}) = \prod_{j=i+1}^t \alpha_j \quad (31)$$

其中 α_j 通过一个线性层参数化，随后在隐藏状态 $\mathbf{x}_j$ 上应用 sigmoid 函数。一项更近期的研究，FoX [21]，也将门控机制融入了 Softmax 注意力，采用了一种本质上与我们在此提出的公式完全相同的表述。有趣的是，注意力机制中的门控通常被解释为一种遗忘机制，因此通常表示为 0 到 1 之间门控值的累积乘积（通过 sigmoid 函数实现）。然而，观察前馈网络中的门控，我们注意到缺乏这种累积机制。我们推测，将门控函数简单地定义为 $g_i(\mathbf{x}_{i:t}) = \mathbf{W}_G(\mathbf{x}_i)^\top \mathbf{x}_t$ 可能实现类似的效果，同时实现起来更简单。我们将这一假设的验证留待未来工作。至于门控机制为何普遍有效，我们鼓励对其基本原理和理论解释进行进一步的研究和探索。

2.2 Memory Update

How is associative memory updated? 例如，通过重新表述方程 1，我们得到：

$$\begin{aligned} \mathbf{S}_t &= \underbrace{\left(\sum_{i=1}^{t-1} \mathbf{v}_i \mathbf{k}_i^\top \right)}_{\mathbf{S}_{t-1}} + \mathbf{v}_t \mathbf{k}_t^\top \\ &= \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top. \end{aligned} \quad (32)$$

这表明，对于经典联想记忆，时间步 t 的更新是通过直接添加最新向量 $\mathbf{v}_t$ 和 $\mathbf{k}_t$ 的外积来实现的。我们通常将方程 32 称为经典联想记忆的循环形式。如果我们将此方程解释为对 $\mathbf{S}$ 的单步梯度下降更新（忽略学习率），则有：

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \underbrace{(-\mathbf{v}_t \mathbf{k}_t^\top)}_{\frac{\partial \mathcal{L}_t}{\partial \mathbf{S}_{t-1}}}, \quad (33)$$

其中 $\mathcal{L}_t$ 表示对应于记忆更新的优化目标，可以显式推导如下：

$$\mathcal{L}_t(\mathbf{S}_{t-1}) = -\langle \mathbf{S}_{t-1} \mathbf{k}_t, \mathbf{v}_t \rangle, \quad (34)$$

其中 $\langle \cdot, \cdot \rangle$ 表示内积。直观上，该目标旨在更新 $\mathbf{S}$ ，使得使用 $\mathbf{v}_t$ 从更新后的记忆中回忆 $\mathbf{k}_t$ 尽可能有效。然而，该目标的一个明显缺点是，除了促进 $\mathbf{S} \mathbf{k}_t$ 和 $\mathbf{v}_t$ 之间的内积增大外，它还鼓励 $\mathbf{S} \mathbf{k}_t$ 的范数无界增长，可能导致数值不稳定。在实践中，可以通过引入例如对 $\mathbf{S}$ [47] 的衰减机制来缓解此问题。下文将详细讨论各种改进的记忆更新方法。

为了便于讨论不同的记忆更新方法，我们首先定义联想记忆的一般循环形式如下：

$$\mathbf{S}_t = \mathbf{A}_t \mathbf{S}_{t-1} \mathbf{B}_t + \mathbf{C}_t, \quad (35)$$

其中 $\mathbf{A}_t$ 、 $\mathbf{B}_t$ 和 $\mathbf{C}_t$ 是参数矩阵。例如，当 $\mathbf{A}_t = \mathbf{I}$ 、 $\mathbf{B}_t = \mathbf{I}$ 和 $\mathbf{C}_t = \mathbf{v}_t \mathbf{k}_t^\top$ 时，方程 32 是方程 35 的一个特例。方程 35 对应什么优化目标？参考附录 B.1，我们可以构建相关的目标函数为：

$$\mathcal{L}_t(\mathbf{S}_{t-1}) = \frac{1}{2} \text{tr}(\mathbf{S}_{t-1}^\top \mathbf{S}_{t-1}) - \frac{1}{2} \text{tr}(\mathbf{S}_{t-1}^\top \mathbf{A}_t \mathbf{S}_{t-1} \mathbf{B}_t) - \text{tr}(\mathbf{C}_t^\top \mathbf{S}_{t-1}), \quad (36)$$

其中 $\text{tr}(\cdot)$ 表示迹运算。注意此构造有一个约束：矩阵 $\mathbf{A}_t$ 和 $\mathbf{B}_t$ 必须对称。通过指定 $\mathbf{A}_t$ 、 $\mathbf{B}_t$ 和 $\mathbf{C}_t$ 的不同组合，我们得到不同的循环形式和相应的优化目标。每种组合对应一种新颖的联想记忆模型，使我们能够从记忆更新的角度明确地解释这些模型。

表 2 总结了各种联想记忆模型、它们在循环形式（公式 35）中对应的参数矩阵（ $\mathbf{A}_t$ 、 $\mathbf{B}_t$ 和 $\mathbf{C}_t$ ）以及优化目标 $\mathcal{L}_t(\mathbf{S}_{t-1})$ 。

Table 2 记忆更新的不同形式。不同的联想记忆模型、它们在循环形式（方程 35）中对应的参数矩阵（ $\mathbf{A}_t$ 、 $\mathbf{B}_t$ 和 $\mathbf{C}_t$ ）以及优化目标 $\mathcal{L}_t(\mathbf{S}_{t-1})$ 。详细推导见附录 B.2。

Model	$\mathbf{A}_t$	$\mathbf{B}_t$	$\mathbf{C}_t$	$\mathcal{L}_t(\mathbf{S}_{t-1})$
Linear Attention [17]	$\mathbf{I}$	$\mathbf{I}$	$\mathbf{v}_t \mathbf{k}_t^\top$	$-\langle \mathbf{S}_{t-1} \mathbf{k}_t, \mathbf{v}_t \rangle$
Gated Linear Attention [47]	$\text{diag}(\boldsymbol{\lambda}_t)$	$\mathbf{I}$	$\mathbf{v}_t \mathbf{k}_t^\top$	$-\langle \mathbf{S}_{t-1} \mathbf{k}_t, \mathbf{v}_t \rangle + \frac{1}{2} \ \text{diag}(\sqrt{1 - \boldsymbol{\lambda}_t}) \mathbf{S}_{t-1}\ _F^2$
DeltaNet [33, 48]	$\mathbf{I}$	$\mathbf{I} - \mathbf{k}_t \mathbf{k}_t^\top$	$\mathbf{v}_t \mathbf{k}_t^\top$	$\frac{1}{2} \ \mathbf{S}_{t-1} \mathbf{k}_t - \mathbf{v}_t\ ^2$
DeltaNet + Momentum	$\mathbf{I}$	$\mathbf{I} - \mathbf{k}_t \mathbf{k}_t^\top$	$\eta_t \mathbf{C}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top = \sum_{i=1}^t \left(\prod_{j=i+1}^t \eta_j \right) \mathbf{v}_i \mathbf{k}_i^\top$	$\frac{1}{2} \ \mathbf{S}_{t-1} \mathbf{k}_t\ ^2 - \sum_{i=1}^t \left(\prod_{j=i+1}^t \eta_j \right) \mathbf{v}_i^\top \mathbf{S}_{t-1} \mathbf{k}_i$
Softmax Attention w/o Norm	$\mathbf{I}$	$\mathbf{I}$	$\mathbf{v}_t \phi(\mathbf{k}_t)^\top$	$-\langle \mathbf{S}_{t-1} \phi(\mathbf{k}_t), \mathbf{v}_t \rangle$
Softmax Attention w/ Norm* [43]	$\frac{t-1}{t} \mathbf{I}$	$\mathbf{I}$	$\frac{1}{t} \mathbf{v}_t \phi(\mathbf{k}_t)^\top$	$-\frac{1}{t} \langle \mathbf{S}_{t-1} \phi(\mathbf{k}_t), \mathbf{v}_t \rangle + \frac{1}{2t} \ \mathbf{S}_{t-1}\ _F^2$
Gated Softmax Attention* [21]	$\frac{t-1}{t} \text{diag}(\boldsymbol{\lambda}_t)$	$\mathbf{I}$	$\frac{1}{t} \mathbf{v}_t \phi(\mathbf{k}_t)^\top$	$-\frac{1}{t} \langle \mathbf{S}_{t-1} \phi(\mathbf{k}_t), \mathbf{v}_t \rangle + \frac{1}{2t} \ \text{diag}(\sqrt{1 - \boldsymbol{\lambda}_t}) \mathbf{S}_{t-1}\ _F^2$

$\boldsymbol{\lambda}$ 与 η 的任意元素均 $\in (0, 1)$ 。

“Softmax Attention” 指标准的因果 softmax 注意力，不包括稀疏 softmax 注意力等变体。“Norm” 表示 softmax 操作的归一化因子：

$\sum_i \exp(\mathbf{q}_t^\top \mathbf{k}_i)$ 。Softmax 注意力的特征映射 $\phi(\cdot)$ 为公式 11。

* t 足够大。

2.2.1 Vignette 2: Softmax Attention with Delta Rule

从表 2 中总结的不同联想记忆模型的比较中，我们能获得哪些洞见？两个核心要点支撑着这些不同的模型：

Softmax 注意力通过引入一个归一化项来解决这个问题，该归一化项大致相当于将新写入的信息 $\mathbf{v}_t$ 按一个衰减因子 $\frac{1}{t}$ 进行缩放。这意味着越接近未来的信息衰减得越厉害，即写入项 $\frac{1}{t} \mathbf{v}_t \mathbf{k}_t^\top \rightarrow 0$ ，从而减缓了 $\mathbf{S}$ 的谱范数的增长。认为越接近未来的信息越不重要这一假设是不合理的。通常，更合理的假设是旧信息的重要性较低。因此，反转衰减方向更有意义。门控线性注意力在其循环形式中通过引入指数衰减因子 $\text{diag}(\boldsymbol{\lambda})$ 来实现这种反向衰减。然而，旧信息并非总是相关性较低。更智能的方法是首先评估新值 $\mathbf{v}_t$ 是否与历史信息重叠，相应地擦除重叠部分，然后将新值写入记忆。这种策略被称为 delta 规则，正是 DeltaNet 所采用的内存更新机制。

受上述讨论启发，can we propose a new associative memory model that integrates the advantages of both the kernel trick and the delta rule? 这样的模型将实现更智能的内存更新管理，同时确保高内存回

忆准确度。

为了实现它，一个直接的方法是将 delta 规则引入 softmax 注意力。如表 2 所示，delta 规则的核心是其循环形式中的衰减项 $\mathbf{I} - \mathbf{k}\mathbf{k}^\top$ 。因此，替换 softmax 注意力循环形式中的 $\mathbf{B}_t$ 并结合核技巧，就能成功地将 delta 规则引入 softmax 注意力：

$$\mathbf{S}_t = \mathbf{S}_{t-1} \underbrace{(\mathbf{I} - \phi(\mathbf{k}_t)\phi(\mathbf{k}_t)^\top)}_{\text{Delta rule decay}} + \mathbf{v}_t\phi(\mathbf{k}_t)^\top. \quad (37)$$

为简化起见，此处省略了 softmax 操作的归一化项，这不影响推导¹⁰。在实践中，归一化项仍然被保留以避免数值不稳定性。

方程 37 看起来复杂。是否可能构建一个联想记忆模型，它完全等价于方程 37 所代表的模型，却拥有更简单的循环形式？理想情况下，循环形式应类似于对齐的 softmax 注意力，即 $\mathbf{A}_t = \mathbf{B}_t = \mathbf{I}$ ，同时 $\mathbf{C}_t$ 保持为外积矩阵。为了实现这一点，我们引入了一个新变量 $\mathbf{u}_t$ ，它依赖于历史的 $\{\mathbf{k}\}_{i=1}^t$ 和 $\{\mathbf{v}\}_{i=1}^t$ 。因此，我们构建了一个新的循环公式：

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{u}_t\phi(\mathbf{k}_t)^\top. \quad (38)$$

为确保该记忆模型与由式 37 所定义的模型等价，我们使用待定系数法求解 $\mathbf{u}_t$ （详见附录 C）。由此得到关于 $\mathbf{u}_t$ 的如下条件：

$$\begin{aligned} \mathbf{u}_t &= - \sum_{i=1}^{t-1} \phi(\mathbf{k}_i)^\top \phi(\mathbf{k}_t) \mathbf{u}_i + \mathbf{v}_t \\ &= - \sum_{i=1}^{t-1} \exp(\mathbf{k}_i^\top \mathbf{k}_t) \mathbf{u}_i + \mathbf{v}_t. \end{aligned} \quad (39)$$

与 softmax 注意力相比， $\mathbf{v}_t$ 现在被 $\mathbf{u}_t$ “取代”，在与 $\phi(\mathbf{k}_t)$ 进行外积后写入记忆。直观上，从式 39 可知， $\mathbf{u}_t$ 是通过一个额外的“处理”步骤从 $\mathbf{v}_t$ 推导出来的。具体而言，首先计算 $\mathbf{k}_t$ 与历史键的相似度，以加权历史 $\{\mathbf{u}_i\}_{i=1}^{t-1}$ 的和，得到 $-\sum_{i=1}^{t-1} \exp(\mathbf{k}_i^\top \mathbf{k}_t) \mathbf{u}_i$ 。该项代表了记忆中与 $\mathbf{v}_t$ 重叠的信息。我们将这一“冗余”信息从 $\mathbf{v}_t$ 中移除，从而获得 $\mathbf{u}_t$ 。此过程正是 delta 规则。

在记忆 $\mathbf{S}$ 更新后，相应的记忆检索方法仍与 softmax 注意力的形式保持一致：

$$\mathbf{o}_t = \mathbf{S}_t \phi(\mathbf{q}_t) = \sum_{i=1}^t \exp(\mathbf{q}_t^\top \mathbf{k}_i) \mathbf{u}_i. \quad (40)$$

总之，与 softmax 注意力相比，新模型在形式上变化极小，仅仅是根据 delta 规则经过一个额外的“处理”步骤后，将原始的 $\mathbf{v}$ 替换为 $\mathbf{u}$ 。通过这一微小修改，我们成功地将 delta 规则融入了 softmax 注意力。

¹⁰如表 2 所示，当 t 变得足够大时，softmax 操作的归一化项会按一个与 $\frac{1}{t}$ 相关的因子缩放循环形式中的 $\mathbf{A}_t$ 和 $\mathbf{C}_t$ 。这个缩放因子是独立的，不影响主要推导。

General Form. 实际上，我们可以通过修改以下组件，以更一般的形式构建新模型：

总之，新模型的一般形式可写为：

$$\mathbf{u}_t = \alpha_t \mathbf{v}_t - \beta_t \sum_{i=1}^{t-1} \kappa_1(\mathbf{k}_i, \mathbf{w}_t) \mathbf{u}_i, \quad (41)$$

$$\mathbf{o}_t = \sum_{i=1}^t \kappa_2(\mathbf{k}_i, \mathbf{q}_t) \mathbf{u}_i, \quad (42)$$

其中 $\kappa_1(\cdot, \cdot)$ 和 $\kappa_2(\cdot, \cdot)$ 是独立的核函数； α_t 和 β_t 是独立的门控； $\mathbf{w}_t$ 是新引入的检索向量。我们将以此一般形式为特征的模型称为 DeltaFormer。

事实上，Transformer 和 DeltaNet 都是上述一般形式的特例。具体来说，若我们设 $\beta_t = 0$ 、 $\alpha_t = 1$ ，并使用 $\mathbf{o}_t = \frac{\sum_{i=1}^t \exp(\mathbf{k}_i^\top \mathbf{q}_t / \sqrt{d}) \mathbf{u}_i}{\sum_{i=1}^t \exp(\mathbf{k}_i^\top \mathbf{q}_t / \sqrt{d})}$ ，则恢复为标准 Transformer。若我们设 $\alpha_t = 1$ 、 $\mathbf{w}_t = \mathbf{k}_t$ ，且令 $\kappa_1(\cdot, \cdot)$ 和 $\kappa_2(\cdot, \cdot)$ 均为线性内积核，则恰好恢复为 DeltaNet。

Model Expressivity. 直观上，神经网络可被视为电路。其表达能力可通过电路复杂度来衡量。电路复杂度研究解决计算问题所需的最小电路规模或深度，从而刻画其中固有的计算难度。

若我们能确定与给定神经网络相对应的电路复杂度，便可判定该网络能够解决的计算问题类别，从而刻画其表达能力。电路复杂性通常关注布尔电路。布尔电路是由逻辑门组成的有向无环图。它由四个参数定义：1) 规模，即电路中门的总数；2) 深度，即从输入到输出的最长路径上的门数；3) 扇入，即每个门可接受的最大输入数（可以是常数或无限制）；以及 4) 门类型：通常为与门、或门、非门。

根据这些参数的不同组合，电路通常分为三类： NC （Nick 类）、 AC （交替类）和 TC （阈值类）。电路深度表征了并行性¹¹且很重要。它通过上标 k 明确表示。例如， NC^k 表示深度为 $\mathcal{O}(\log^k(n))$ 的电路，其中 n 是输入长度。

我们提供这三类电路的形式定义：

由于扇入限制的差异（ AC 中为无限扇入，而 NC 中为有限扇入）以及 TC 中阈值门相较于 AC 具有更强的表达能力，我们得到以下三类电路之间的层次关系（亦见图 1.）：

$$NC^0 \subset AC^0 \subset TC^0 \subseteq NC^1 \subseteq AC^1 \subseteq TC^1 \subseteq NC^2 \subseteq \dots \subseteq P. \quad (43)$$

为了直观地理解这三类电路之间的关系，考虑两个说明性示例：

那么，Transformer 对应的电路复杂性是什么？给定序列长度 n ，Merrill and Sabharwal [23] 已经表明，没有思维链推理的恒定深度 Transformer 只能解决 TC^0 内的问题。同时，最近的几项工作 [13, 24, 38] 证明 DeltaNet 的表达能力超过了 TC^0 。

通过证明本节提出的 DeltaFormer 能够执行 n 个元素的追踪¹²，对于 $n \geq 5$ （详见附录 E），the expressivity of DeltaFormer surpasses TC^0 and theoretically reaches NC^1 .

¹¹直观上，对于单层电路，输出可以一步计算出来。然而，如果电路有 k 层，则需要 k 个顺序步骤才能获得输出。这意味着电路层数与并行度成反比。

¹²追踪 n 个状态本质上指的是在长度为 n 的序列内交换元素的任务。

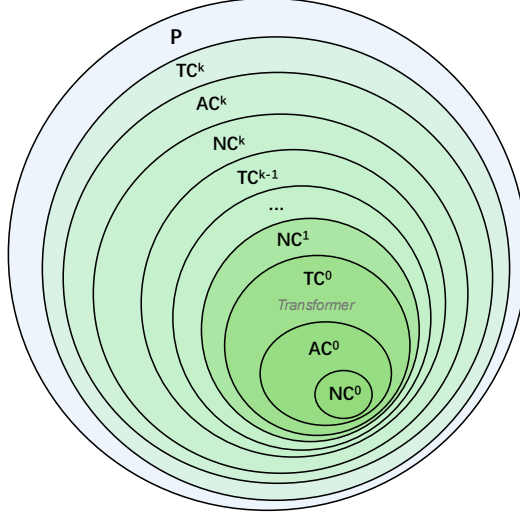


Figure 1 不同电路类之间的层次关系。三类电路 NC 、 AC 和 TC 可解决的问题的复杂性包含在 P 内。具体来说，我们有： $NC^0 \subset AC^0 \subset TC^0 \subseteq NC^1 \subseteq AC^1 \subseteq TC^1 \subseteq NC^2 \subseteq \dots \subseteq P$ 。

Algorithm Implementation. 我们首先分析算法的复杂性。令向量 $q, k, v, u \in \mathbb{R}^d$ ¹³，且 T 为序列长度。

根据上述分析， $\mathcal{O}(T)$ 的训练阶段时间复杂度明显差于 softmax 注意力的 $\mathcal{O}(1)$ 复杂度。因此，提升训练并行性至关重要。为解决此问题，我们可以将序列划分为多个块，并力求在每个块内实现完全并行计算。因此，整体训练时间复杂度可优化至 $\mathcal{O}(T/C)$ ，其中 C 表示块大小。具体实现细节和伪代码可参见附录 D.3。此外，我们在附录 E.4 中提供了 DeltaFormer 状态跟踪玩具实验的参考代码。

2.2.2 Vignette 3: What Will Happen in Infinitely Long In-Context Learning?

如表 2 所示，不同的联想记忆模型对应着不同的记忆更新优化目标。以 softmax 注意力为例，在测试时解码——即上下文学习（ICL）——过程中，对应的记忆 S 会经历单步梯度下降更新。考虑一个极端场景：as the length of ICL extends indefinitely, what would be the behavior of the entire optimization process? 探究这一极限情况或许有助于预测为极长上下文设计的模型的技术演进。

Single-head DeltaNet. 具体而言，我们首先考虑一个具有代表性的简单模型：单头注意力 DeltaNet¹⁴。DeltaNet 的优化目标为：

$$\mathcal{L}_t(S_{t-1}) = \frac{1}{2} \|S_{t-1}k_t - v_t\|^2. \quad (44)$$

让我们尝试推导与此优化目标对应的最优记忆 S^* ：

$$S^* = \arg \min_S \mathbb{E}_x \left[\frac{1}{2} \|SW_k x - W_v x\|^2 \right], \quad (45)$$

¹³对于内积， q 和 k 必须具有相同的维度。式 39 意味着 v 和 u 也具有相同的维度。虽然 k 和 v 的维度可能不同，但这种差异通常受常数因子限制。为简单起见，我们在此假设它们的维度相等。

¹⁴原始线性注意力对应的优化目标是无界的。随着 t 增大， S_k 的范数会爆炸，导致模型崩溃。这意味着线性注意力（未经记忆衰减等优化）无法进行非常长的上下文学习。DeltaNet 的优化目标是有界的，避免了上述问题，使其成为更合理的分析与比较起点。

其中 $\mathbf{x}$ 表示输入变量, $\mathbf{k} = \mathbf{W}_k \mathbf{x}$ 、 $\mathbf{v} = \mathbf{W}_v \mathbf{x}$ 和 $\{\mathbf{W}_k, \mathbf{W}_v\}$ 是测试时固定的模型权重。根据附录 F, $\mathbf{S}^*$ 的解析解为:

$$\mathbf{S}^* = (\mathbf{W}_v \mathbb{E}_{\mathbf{x}}[\mathbf{x}\mathbf{x}^\top] \mathbf{W}_k^\top) (\mathbf{W}_k \mathbb{E}_{\mathbf{x}}[\mathbf{x}\mathbf{x}^\top] \mathbf{W}_k^\top)^{-1}. \quad (46)$$

由于 $\mathbf{W}_k$ 通常是可逆的¹⁵且 $\mathbb{E}_{\mathbf{x}}[\mathbf{x}\mathbf{x}^\top]$ 通常是正定的。因此, 解析解可简化为:

$$\mathbf{S}^* = \mathbf{W}_v \mathbf{W}_k^{-1}. \quad (47)$$

这表明, 理想情况下, 随着记忆更新的进行, 记忆 $\mathbf{S}$ 会收敛到固定值 $\mathbf{S}^*$ 。当记忆不再更新时, 意味着没有新信息写入记忆, 回忆结果完全取决于固定的 $\mathbf{S}^*$ 。若将 $\mathbf{S}^*$ 视为模型权重的一部分, 单头 DeltaNet 便退化为一个二元语法模型, 其输出仅由当前输入决定。Theoretically, as the context length approaches infinity, the single-head DeltaNet loses its in-context learning capability.

Multi-head DeltaNet. 在多头设置的情况下, $\mathbf{W}_k$ 和 $\mathbf{W}_v$ 变为低秩矩阵。因此, 公式 46 被简化为:

$$\mathbf{S}^* = \mathbf{W}_v \mathbb{E}_{\mathbf{x}}[\mathbf{x}\mathbf{x}^\top] \mathbf{W}_k^+ \mathbf{W}_k \mathbb{E}_{\mathbf{x}}[\mathbf{x}\mathbf{x}^\top]^{-1} \mathbf{W}_k^+, \quad (48)$$

其中 $\mathbf{W}_k^+$ 表示 $\mathbf{W}_k$ 的伪逆。由于通常 $\mathbf{W}_k^+ \mathbf{W}_k \neq \mathbf{I}$, 项 $\mathbb{E}_{\mathbf{x}}[\mathbf{x}\mathbf{x}^\top]$ 无法被消除。这意味着最优记忆 $\mathbf{S}^*$ 依赖于数据分布, 可能不是固定的。The low-rank property of projection matrices (e.g., $\mathbf{W}_k$) 可能缓解模型上下文学习能力的退化。

Softmax Attention. 鉴于多头结构可以防止 ICL 退化, 这是否意味着在构建支持无限上下文长度的模型方面已无理论障碍? 至少对于 softmax 注意力而言, 情况并非如此。

根据表 2, softmax 注意力的优化目标是:

$$\mathcal{L}_t(\mathbf{S}_{t-1}) = \frac{1}{t} \left(\frac{1}{2} \|\mathbf{S}_{t-1}\|_F^2 - \langle \mathbf{S}_{t-1} \phi(\mathbf{k}_t), \mathbf{v}_t \rangle \right). \quad (49)$$

当 t 足够大时, 由于 $\frac{1}{t}$ 项的存在, 我们有 $\mathcal{L}_t(\mathbf{S}_{t-1}) \rightarrow 0$, 导致梯度趋近于零, 且 $\mathbf{S}$ 逐渐停止更新。This implies that the ICL capability of softmax attention will still gradually diminish as the context length increases. 直观上, 当上下文 (即 KV 缓存) 变得过长时, 存储在上下文中的信息类似于一个 “广阔的海洋”, 信噪比很低, 使得精确回忆目标信息变得困难, 从而导致 ICL 不可行。

如果我们移除归一化, 尽管梯度不会随着 t 增大而消失, 但未经归一化的 softmax 注意力对应着一个无界的优化目标, 最终将导致模型崩溃。

Summary. 关联记忆模型在理论上存在随着序列长度增加而逐渐收敛到一个固定值的风险。具体到注意力层, 记忆倾向于逐步停止更新, 导致 ICL 能力下降。因此, constructing models capable of infinite contexts may be challenging. 但必须承认, 实际场景要复杂得多: 诸如多头注意力、记忆衰减和短键值卷积等机制可以有效防止记忆更新停滞。此外, 大型神经网络通常具有多层结构, 其中单层优化中的波

¹⁵方阵通常是满秩的, 因此可逆。

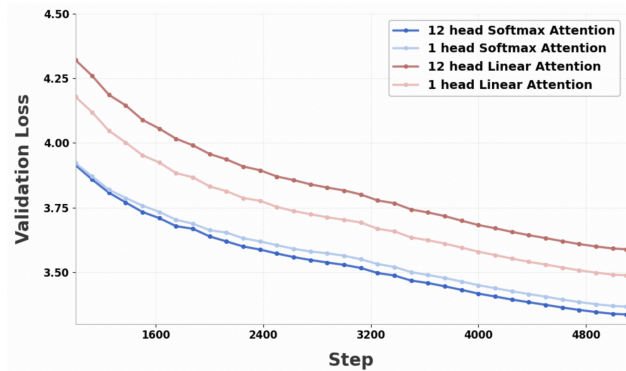


Figure 2 不同注意力机制下维度的影响。使用 GPT-2 小模型（12 层，768 维）的验证损失。减少头数有利于线性注意力，但会损害 Softmax 注意力，这很可能源于二者在检索和表达能力上的差异。

动（例如由 $\mathbb{E}_{\mathbf{x}}[\mathbf{x}\mathbf{x}^T]$ 引入的变异性）可能在层间累积并放大，从而阻碍记忆平滑地接近理论解析解。

3 Experiments

3.1 Multi-head Efficiency Trade-off

我们在一个具有 12 层和 768 个隐藏维度的 GPT-2 小 Transformer 模型上进行实验，以研究注意力机制如何响应头数与每头维度的变化。我们的设置遵循开源 BlaGPT 实现¹⁶，包括数据集和训练配置。

对于线性注意力设置，我们仅将 Softmax 操作替换为线性核函数，并应用每头 RMS 归一化以保持训练稳定性。为确保公平比较，我们也从 Softmax 基线中移除了原有的归一化项，并改用每头 RMS 归一化。

如图 2 所示，线性注意力显著受益于将头数减少至 1，从而增加每头维度，而 Softmax 注意力在相同条件下性能下降。这支持了我们的假设：Softmax 更依赖于通过多头机制获得的表达能力，而线性注意力则从更大的记忆容量中获益更多。这些发现为第 2.1 节中的假设提供了初步的实证验证。

3.2 Track the Exchange of Elements

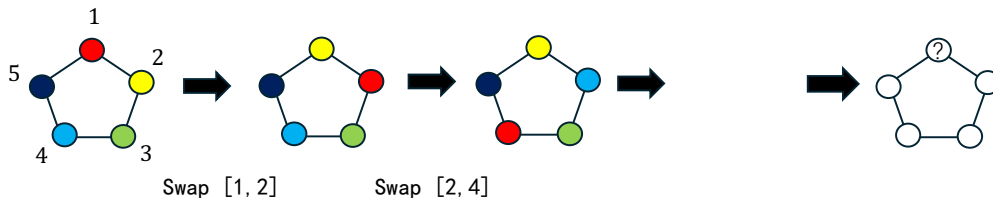


Figure 3 交换任务示意图。初始时，不同颜色的令牌被放置在位置 1 到 5，每一步交换两个位置的令牌。我们期望模型在每一步查询每个位置的令牌是什么。为简化但不失一般性，我们默认输出第一个位置的令牌，以避免引入“查询令牌”。此任务也可被分词为输入词汇表大小为 $C_5^2 = 10$ 、输出词汇表大小为 5 的任务。

尽管我们的定理 1 证明了第 2.2.1 节中描述的 DeltaFormer 能够追踪 n 个对象的交换，但这仍需通过实验验证。通过使用梯度下降，DeltaFormer 能否从数据中学会追踪 n 个对象的交换？为此，我们在本节进行实验以验证这一点。设置如图 3 所示，默认上下文长度为 16。¹⁷

¹⁶<https://github.com/erogol/BlaGPT>

¹⁷为简化起见，本节省略了相似性函数中使用的温度系数。

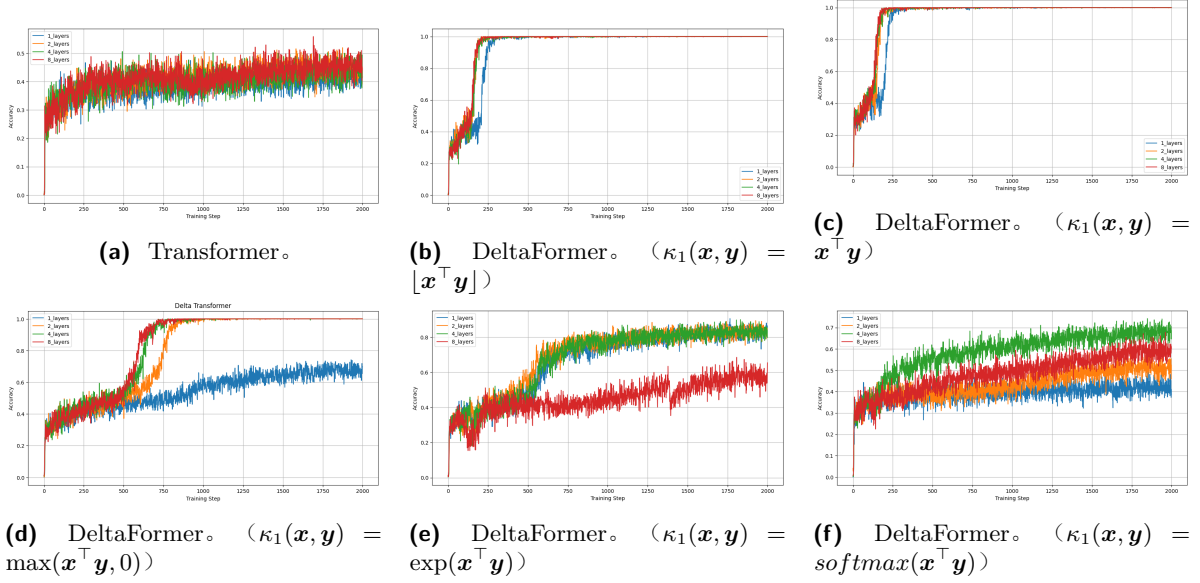


Figure 4 使用不同相似性函数 $\kappa_1(\cdot)$ 执行交换任务时, Transformer 与 DeltaFormer 的比较。对于 $\kappa_2(\cdot)$, 我们使用 softmax 函数以保持与 Transformer 的一致性。请注意 y 轴的比例。为确保收敛, $[\cdot]$ 表示四舍五入到两位小数, 例如 $[1.236] = 1.24$ 。

DeltaFormer Can Track the Exchange of Elements. 我们在不同相似性函数设计下比较了 DeltaFormer 和 Transformer, 如图 4 所示。在几乎所有可想象的简单 $\kappa_1(\cdot)$ 设计下, DeltaFormer 都取得了比 Transformer 模型更好的结果。并且单层 DeltaFormer 能够执行并追踪 5 个元素的交换操作。但增加 Transformer 的层数并未带来任何改进。我们推测这可能涉及优化问题, 即使是对于 DeltaFormer 也是如此。我们将在后续章节中探讨这一假设。**The Kernel κ_1 Used in u_t 是重要的。**另一个值得注意的点是, 不同相似度函数的选择也对最终效果有显著影响。根据我们在定理 1 中的构造性证明, 该相似度函数能够有效追踪 5 个元素。相似度函数越接近构造方法, 其性能越好。softmax 的归一化项对 exp 的相似度计算有负面影响。而我们阅读方法中使用的 κ_2 并非基于构造的相似度来确保其与标准注意力相同, 而是使用了 softmax。即便如此, 一个合适的 κ_1 可以达到 100% 的效果。因为我们的定理 1 实际上证明了存在一种方法, 能以某种形式的 u 读取每个位置上的元素。这意味着元素的交换隐含地包含在 u 的更新中。直观地说, 如果 κ_1 的相似度选择不当, 会导致 u 的更新产生更多的累积误差。从数学上讲, 这实际上反映了一个逆矩阵的扰动很可能是不适定的。我们将 u 和 o 的计算重写如下:

$$\begin{aligned} u &= A_1^{-1}v \\ o &= A_2u, \end{aligned} \tag{50}$$

那么我们将得到:

$$\|(A_1 + \Delta A)^{-1}V - A_1^{-1}V\| \approx \|A_1^{-1}(\Delta A)A_1^{-1}V\| \leq \|A_1^{-1}\| \|\Delta A\| \|A_1^{-1}\| \|V\| = \|A_1^{-1}\|^2 \|\Delta A\| \|V\|, \tag{51}$$

以及

$$\|(\mathbf{A}_2 + \Delta \mathbf{A}_2)\mathbf{U} - \mathbf{A}_2\mathbf{U}\| = \|(\Delta \mathbf{A}_2)\mathbf{U}\| \leq \|\Delta \mathbf{A}_2\| \|\mathbf{U}\|. \quad (52)$$

$\mathbf{u}$ 的计算稳定性弱于 $\mathbf{o}$ 的计算，因此 $\mathbf{u}$ 的计算需要在稳定性和表达能力之间取得平衡，我们提供了一种类似分组查询注意力的方法来平衡表达能力和训练稳定性，如附录 E.2 所示。

Stress Testing of Linear and Nonlinear Kernels. 我们也在本节中对圆形和线性函数进行了压力测试，其中 $d = 128$ 。设置类似于第 3.2 节，但实验条件是 n 大于或等于 128。我们使用 $n \in \{128, 256, 512\}$ ，相应的训练长度为 $\{256, 512, 1024\}$ ，以尽可能确保大多数元素参与交换。此外，为了避免优化问题，我们采用了定理 1 中使用的几乎正交向量来设置模型的键和值，模型只需要学习从状态空间中读取信息。结果如图 5 所示。我们可以观察到，当 d 固定时，随着 $n \geq d$ 的增加，线性核的性能严重下降。这本质上涉及著名的汤普森问题，即如何在 d 维单位球面上放置尽可能多的正交向量。然而线性函数无法进行叠加，而非线性函数可以通过叠加存储大量信息 [9]。

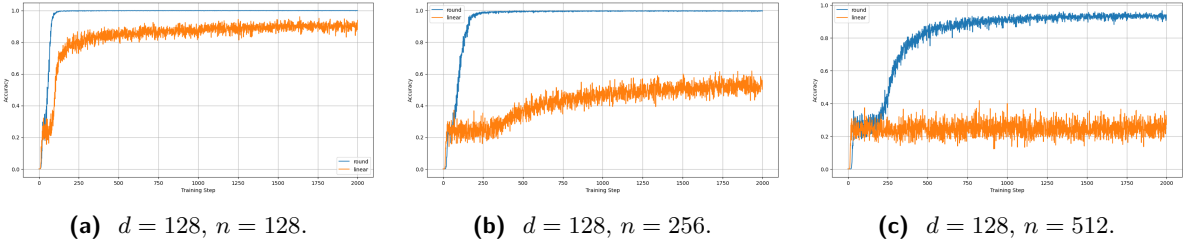


Figure 5 DeltaFormer 与 $\kappa(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{y}$ 和 $\kappa(\mathbf{x}, \mathbf{y}) = [\mathbf{x}^\top \mathbf{y}]$ 的比较。

Curriculum Learning Is Important. 如图所示，我们最初在长度为 256 的情况下进行训练，模型的收敛速度非常慢。因此我们决定从 32 开始逐步延长窗口，即逐步增加难度。我们发现，在这种课程学习的基础上，模型可以用更少的计算和样本获得更好的性能。在使用 Transformer 进行上下文学习时也观察到了类似的现象 [12]。

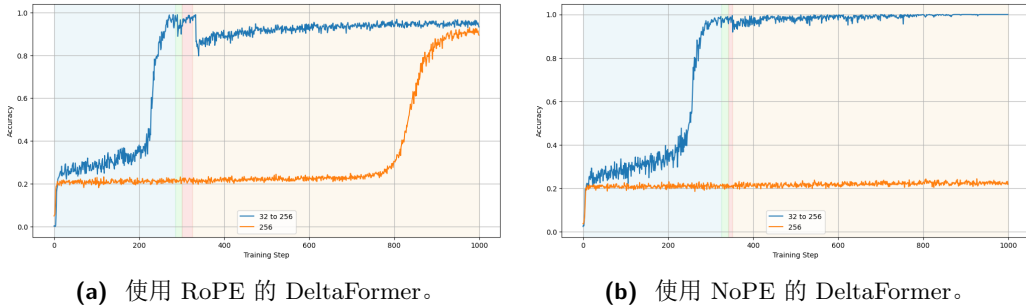


Figure 6 使用不同学习策略和位置嵌入的 DeltaFormer 的比较。均使用 $\kappa_1(\mathbf{x}, \mathbf{y}) = [\mathbf{x}^\top \mathbf{y}]$ 。“32 到 256”表示初始训练长度为 32，即交换次数为 32。当准确率达到 0.99 时，训练长度将加倍，直至达到 256。图像中的每种颜色渐变代表训练长度的一次加倍。而“256”表示模型从一开始就在训练长度 256 上进行训练。y 轴反映了当前训练长度下的准确率。

The Role of Rotary Embeddings [39]. 因为我们定理 1 的证明不需要位置嵌入。因此，另一个有趣的实验是，我们移除了现代 Transformer 中广泛使用的默认旋转位置嵌入。我们发现，移除位置嵌入后，模型的收敛速度变慢，甚至当直接在长度 256 上训练时，模型得到了随机分数。另一方面，移除 RoPE 后，“32

到 256” 能够达到 100% 的准确率。并且在扩展过程中，NoPE 在跳跃点处的性能下降减小，表明 NoPE 具有更好的长度泛化能力。也有一些研究 [18] 关于位置嵌入的长度泛化能力得出了类似的发现。因此，我们推测 RoPE 可能损害了模型的表达能力和外推能力，但它更有利于优化。

3.3 Reachability of Directed Acyclic Graphs

此外，我们有一个简单的图连通性任务，涉及在有向无环图上确定可达性。为简化起见，我们仅考虑在某种拓扑排序中，其他节点能否被第一个节点到达。并且每个节点在初始时最多编码其相邻节点的信息。由于最终输出为真或假，为避免类别不平衡，我们通过将 n 个点平均划分为 2 个类别来构建输入数据，每个类别对应一棵树。因此，我们仅判断从某个根节点出发到其他节点的可达性。接着，我们只对每个节点编码其父节点信息。

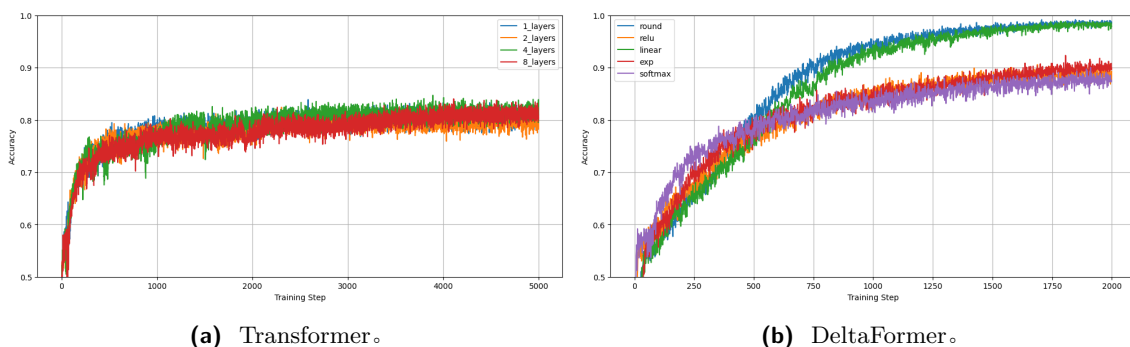


Figure 7 使用不同相似度函数 $\kappa_1(\cdot)$ 执行有向无环图可达性任务时，Transformer 与 DeltaFormer 的比较。对于 $\kappa_2(\cdot)$ ，我们使用 softmax 函数以保持与标准 Transformer 的一致性。

DeltaFormer Performs Better Than Transformer. 我们在 32 个节点上进行了实验，如图 7 所示。多层 Transformer 模型也难以达到 100% 的准确率，但单层 DeltaFormer 可以表现得非常好。理论上，Transformer 需要 $\mathcal{O}(\log n)$ 层来对 n 个节点执行连通性检查 [32]。然而，基于图 7a，我们推测优化问题也限制了 Transformer 在此任务上的有效性。

The Power of Matrix Inverse. 如公式 97 所示， u 的计算可以通过矩阵求逆来重写。如果已知邻接矩阵 A ，那么要判断节点 i 和 j 是否连通，只需依次计算 $A, A^2, A^3, \dots, A^n$ ，然后观察该矩阵的 (i, j) 元素是否大于 0。而 $(I - A)^{-1}$ 可视为 $I + A + \dots + A^n$ 的近似。因此，矩阵求逆操作极大地提升了模型的表达能力。我们可以预期，具备矩阵求逆能力的模型在图相关任务中能取得更好的性能。Fagnou et al. [10] 也利用矩阵求逆进行高效的实体追踪。本质上，矩阵求逆这一操作，其能力远超 TC^0 ，增强了 Transformer 的表达力。

4 Conclusion

从关联记忆的棱镜审视 Transformer 的这番探索，仅仅是我们持续追寻以揭示这些强大架构全部潜力与复杂运作过程中的一小步。尽管我们相信这一视角为理解记忆容量与更新动态等现象提供了宝贵的清晰度与统一框架，它同时也开启了一片充满更多问题与深入探索途径的广阔天地。我们尤其对以下基本问题感到着迷，并渴望进一步探究：

The Architectural Endgame: 是否存在一种终极的、最优的模型架构设计？我们所讨论的关联记忆，是否

是实现人工智能最有效的范式？如果这一框架确实处于核心地位，那么我们当前具有独立注意力层与前馈层的 Transformer 结构，是正在接近一个根本性的最优状态，还是存在着截然不同、却更强大的关联记忆实现方式等待被发现？

« $\langle \text{LIX}_C \text{MD}_2 \rangle \rangle \rangle$ *Transformer*

Parallelism, Hardware, and the Next Leap in Intelligence: 现有 Transformer 模型固有的并行计算复杂度（主要处于 TC^0 内）限制了它们能够高效学习和解决的问题类型。这一限制与我们当前并行硬件的性能本质相关。追求具有更高并行复杂度类别（例如 TC^1 甚至 P 完全任务）的模型是否具有重大价值？下一代智能的出现是否依赖于硬件的范式转变——或许是量子计算等技术的成熟，这些技术理论上承诺能高效解决当前在 TC^k 内难以处理的问题？

这些问题凸显了我们面前依然广阔而肥沃的领域。理解与复制智能的旅程远未结束。我们找到的每一个答案往往照亮了十几个新的问题，每一个都比前一个更为深刻。我们希望通过提出这些挑战并分享我们当前的理解，能够激发研究界内进一步的讨论、协作与创新，共同推动人工智能所能实现的边界。前方的道路是复杂的，但对这些根本性问题的追求，预示着一个充满发现的丰饶未来。**Bridging Contextual and Persistent Memory**

5 Acknowledgements

我们衷心感谢杨松林、廖本成、魏来和余翔宇在本工作发展过程中提供的宝贵反馈和深刻见解。同时，我们感谢刘世龙在与 Deltanet 相关的实验支持方面提供的帮助，以及冯古浩在电路复杂性理论方面富有启发性的讨论。我们诚挚感谢蔡天乐对许多公式推导和高层概念思考提出的建设性意见。我们进一步感谢何佳傲、卢建桥和姚和源在算法设计与代码级实验等各个实现环节给予的支持。上文及作者列表中提及的部分姓名为公司内部使用的别名。

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.
- [2] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. arXiv preprint arXiv:2501.00663, 2024.
- [3] Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. It’s all connected: A journey through test-time memorization, attentional bias, retention, and online optimization, 2025. URL <https://arxiv.org/abs/2504.13173>.
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. Advances in neural information processing systems, 33:1877–1901, 2020.
- [5] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. arXiv preprint arXiv:2307.08691, 2023.
- [6] Tri Dao, Daniel Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In Advances in Neural Information Processing Systems, volume 35, pages 12556–12570, 2022.
- [7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929, 2020.
- [8] Nelson Elhage, Tristan Hume, Catherine Olsson, Neel Nanda, Tom Henighan, Scott Johnston, Sheer ElShowk, Nicholas Joseph, Nova DasSarma, Ben Mann, Danny Hernandez, Amanda Askell, Kamal Ndousse, Andy Jones, Dawn Drain, Anna Chen, Yuntao Bai, Deep Ganguli, Liane Lovitt, Zac Hatfield-Dodds, Jackson Kernion, Tom Conerly, Shauna Kravec, Stanislav Fort, Saurav Kadavath, Josh Jacobson, Eli Tran-Johnson, Jared Kaplan, Jack Clark, Tom Brown, Sam McCandlish, Dario Amodei, and Christopher Olah. Softmax linear units. Transformer Circuits Thread, 2022. <https://transformer-circuits.pub/2022/solu/index.html>.
- [9] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. Transformer Circuits Thread, 2022. https://transformer-circuits.pub/2022/toy_model/index.html.
- [10] Erwan Fagnou, Paul Caillon, Blaise Delattre, and Alexandre Allauzen. Chain and causal attention for efficient entity tracking. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 13174–13188, 2024.
- [11] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research, 23(120):1–39, 2022.
- [12] Shivam Garg, Dimitris Tsipras, Percy S Liang, and Gregory Valiant. What can transformers learn in-context? a case study of simple function classes. Advances in Neural Information Processing Systems, 35:30583–30598, 2022.
- [13] Riccardo Grazi, Julien Siems, Arber Zela, Jörg KH Franke, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear rnns through negative eigenvalues. arXiv preprint arXiv:2411.12537, 2024.

- [14] Tom Henighan, Shan Carter, Tristan Hume, Nelson Elhage, Robert Lasenby, Stanislav Fort, Nicholas Schiefer, and Christopher Olah. Superposition, memorization, and double descent. Transformer Circuits Thread, 6:24, 2023.
- [15] Geoffrey E Hinton and James A Anderson. Parallel models of associative memory: updated edition. Psychology press, 2014.
- [16] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with alphafold. nature, 596(7873):583–589, 2021.
- [17] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In International conference on machine learning, pages 5156–5165. PMLR, 2020.
- [18] Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers. Advances in Neural Information Processing Systems, 36:24892–24928, 2023.
- [19] Teuvo Kohonen. Correlation matrix memories. IEEE transactions on computers, 100(4):353–359, 1972.
- [20] Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, et al. Minimax-01: Scaling foundation models with lightning attention. arXiv preprint arXiv:2501.08313, 2025.
- [21] Zhixuan Lin, Evgenii Nikishin, Xu Owen He, and Aaron Courville. Forgetting transformer: Softmax attention with a forget gate. arXiv preprint arXiv:2503.02130, 2025.
- [22] Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, et al. Moba: Mixture of block attention for long-context llms. arXiv preprint arXiv:2502.13189, 2025.
- [23] William Merrill and Ashish Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers. Transactions of the Association for Computational Linguistics, 11:531–545, 2023.
- [24] William Merrill, Jackson Petty, and Ashish Sabharwal. The illusion of state in state-space models. In International Conference on Machine Learning, pages 35492–35506. PMLR, 2024.
- [25] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. arXiv preprint arXiv:2209.11895, 2022.
- [26] Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Haowen Hou, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, Saiteja Utpala, et al. Rwkv-7” goose” with expressive dynamic state evolution. arXiv preprint arXiv:2503.14456, 2025.
- [27] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. arXiv preprint arXiv:2309.00071, 2023.
- [28] Michael Poli, Armin W Thomas, Eric Nguyen, Pragaash Ponnusamy, Björn Deiseroth, Kristian Kersting, Taiji Suzuki, Brian Hie, Stefano Ermon, Christopher Re, et al. Mechanistic design and scaling of hybrid architectures. In International Conference on Machine Learning, pages 40908–40950. PMLR, 2024.

- [29] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In International conference on machine learning, pages 8748–8763. PmLR, 2021.
- [30] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In International conference on machine learning, pages 28492–28518. PMLR, 2023.
- [31] Hubert Ramsauer, Bernhard Schäfl, Johannes Lehner, Philipp Seidl, Michael Widrich, Thomas Adler, Lukas Gruber, Markus Holzleitner, Milena Pavlović, Geir Kjetil Sandve, et al. Hopfield networks is all you need. arXiv preprint arXiv:2008.02217, 2020.
- [32] Clayton Sanford, Bahare Fatemi, Ethan Hall, Anton Tsitsulin, Mehran Kazemi, Jonathan Halcrow, Bryan Perozzi, and Vahab Mirrokni. Understanding transformer reasoning capabilities via graph algorithms. Advances in Neural Information Processing Systems, 37:78320–78370, 2024.
- [33] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In International conference on machine learning, pages 9355–9366. PMLR, 2021.
- [34] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. arXiv preprint arXiv:2407.08608, 2024.
- [35] Noam Shazeer. Glu variants improve transformer, 2020. URL <https://arxiv.org/abs/2002.05202>.
- [36] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv preprint arXiv:1701.06538, 2017.
- [37] Sam Shleifer, Jason Weston, and Myle Ott. Normformer: Improved transformer pretraining with extra normalization, 2021. URL <https://arxiv.org/abs/2110.09456>.
- [38] Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. Deltaproduct: Improving state-tracking in linear rnns via householder products. arXiv preprint arXiv:2502.10297, 2025.
- [39] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024.
- [40] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. arXiv preprint arXiv:2307.08621, 2023.
- [41] Wendy A Suzuki. Associative learning and the hippocampus. 2005.
- [42] A Vaswani. Attention is all you need. Advances in Neural Information Processing Systems, 2017.
- [43] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.
- [44] Jie Wang, Tao Ji, Yuanbin Wu, Hang Yan, Tao Gui, Qi Zhang, Xuanjing Huang, and Xiaoling Wang. Length generalization of causal transformers without position encoding. arXiv preprint arXiv:2404.12224, 2024.
- [45] Ke Alexander Wang, Jiaxin Shi, and Emily B Fox. Test-time regression: a unifying framework for designing sequence models with associative memory. arXiv preprint arXiv:2501.12352, 2025.
- [46] Mingyu Xu, Wei Cheng, Bingning Wang, and Weipeng Chen. Kv shifting attention enhances language modeling. arXiv preprint arXiv:2411.19574, 2024.

- [47] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. arXiv preprint arXiv:2312.06635, 2023.
- [48] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. arXiv preprint arXiv:2406.06484, 2024.
- [49] Biao Zhang and Rico Sennrich. Root mean square layer normalization. Advances in Neural Information Processing Systems, 32, 2019.
- [50] Yufei Zhao. Probabilistic methods in combinatorics. Draft available at <https://yufeizhao.com/pm>, 2022.

Appendix

A Derivation of the Inverse SNR Formula

由于关联记忆的线性形式是方程 9 的一个特例，我们首先推导方程 10，然后针对线性和指数核函数将推导过程具体化。

Inverse SNR with kernel. 首先，我们可以将使用核函数 κ 的关联回忆方程写为如下形式：

$$\mathbf{S}_t \phi(\mathbf{k}_i) = \mathbf{v}_i \kappa(\mathbf{k}_i, \mathbf{k}_i) + \sum_{j \neq i} \mathbf{v}_j \kappa(\mathbf{k}_j, \mathbf{k}_i). \quad (53)$$

根据方程 6，令

$$\mathbf{r} = \sum_{j \neq i} \mathbf{v}_j \kappa(\mathbf{k}_j, \mathbf{k}_i), \quad c = \kappa(\mathbf{k}_i, \mathbf{k}_i),$$

则我们有

$$\begin{aligned} \text{SNR}^{-1} &= \mathbb{E}_{\mathbf{v}_j, \mathbf{k}_j} \left[\frac{\|\mathbf{r}\|^2}{c^2 \|\mathbf{v}_i\|^2} \right] \\ &= \frac{\mathbb{E}_{\mathbf{v}_j, \mathbf{k}_j} \left[\left\| \sum_{j \neq i} \mathbf{v}_j \kappa(\mathbf{k}_j, \mathbf{k}_i) \right\|^2 \right]}{c^2 \|\mathbf{v}_i\|^2}. \end{aligned} \quad (54)$$

从方程 54 继续，我们展开期望内的平方项：

$$\begin{aligned} \text{SNR}^{-1} &= \mathbb{E}_{\mathbf{v}_j, \mathbf{k}_j} \left[\frac{\left\| \sum_{j \neq i} \mathbf{v}_j \kappa(\mathbf{k}_j, \mathbf{k}_i) \right\|^2}{c^2 \|\mathbf{v}_i\|^2} \right] \\ &= \frac{1}{c^2 \|\mathbf{v}_i\|^2} \mathbb{E}_{\mathbf{v}_j, \mathbf{k}_j} \left[\left(\sum_{j \neq i} \mathbf{v}_j \kappa(\mathbf{k}_j, \mathbf{k}_i) \right)^\top \left(\sum_{j' \neq i} \mathbf{v}_{j'} \kappa(\mathbf{k}_{j'}, \mathbf{k}_i) \right) \right]. \end{aligned} \quad (55)$$

由于假设向量 $\mathbf{v}_j$ 是独立同分布的标准高斯向量，交叉项 $j \neq j'$ 的期望为零，仅保留对角项。因此，我们得到

$$\begin{aligned} &= \frac{1}{c^2 \|\mathbf{v}_i\|^2} \mathbb{E}_{\mathbf{k}_j} \left[\sum_{j \neq i} \mathbb{E}_{\mathbf{v}_j} [\|\mathbf{v}_j\|^2] \kappa^2(\mathbf{k}_j, \mathbf{k}_i) \right] \\ &= \frac{1}{c^2 \|\mathbf{v}_i\|^2} \mathbb{E}_{\mathbf{k}_j} [(N-1) \mathbb{E}_{\mathbf{v}_j} [\|\mathbf{v}_j\|^2] \kappa^2(\mathbf{k}_j, \mathbf{k}_i)], \end{aligned} \quad (56)$$

其中 N 表示存储键的总数。

假设 $\mathbb{E}[\|\mathbf{v}_j\|^2] = \|\mathbf{v}_i\|^2$ ，这在存储值具有近似相同量级时成立，我们进一步简化：

$$= \frac{(N-1)}{c^2} \mathbb{E}_{\mathbf{k}_j} [\kappa^2(\mathbf{k}_j, \mathbf{k}_i)]. \quad (57)$$

最后，对于大的 N ，我们使用 N 代替 $(N-1)$ 作为缩写，从而得到

$$\text{SNR}^{-1} = N \frac{\mathbb{E}_{\mathbf{k}_j} [\kappa^2(\mathbf{k}_j, \mathbf{k}_i)]}{\kappa^2(\mathbf{k}_i, \mathbf{k}_i)}. \quad (58)$$

Linear kernel. 固定一个探测键 $\mathbf{k}_i \in \mathbb{R}^{d_k}$ 并绘制每个其他键 $\mathbf{k}_j \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{d_k})$ 。对于线性核 $\kappa(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{y}$ ，随机内积 $X_j := \mathbf{k}_j^\top \mathbf{k}_i$ 是通过将 $\mathbf{k}_j$ 投影到 $\mathbf{k}_i$ 的方向上得到的，因此

$$X_j \mid \mathbf{k}_i \sim \mathcal{N}(0, \|\mathbf{k}_i\|_2^2), \quad \text{so} \quad \mathbb{E}[X_j^2] = \|\mathbf{k}_i\|_2^2.$$

将此方差代入定义得到

$$\text{SNR}_{\text{Linear}}^{-1} = N \frac{\mathbb{E}[X_j^2]}{(\mathbf{k}_i^\top \mathbf{k}_i)^2} = \frac{N}{\|\mathbf{k}_i\|_2^2}.$$

因为 $\|\mathbf{k}_i\|_2^2 \sim \chi_{d_k}^2$ 的均值为 d_k 且在 d_k 较大时围绕该值急剧集中，我们将其替换为其期望值以获得高维估计

$$\text{SNR}_{\text{Linear}}^{-1} = \frac{N}{d_k + \mathcal{O}(\sqrt{d_k})}.$$

Exponential kernel. 对于 $\kappa_{\text{exp}}(\mathbf{x}, \mathbf{y}) = \exp(\frac{\mathbf{x}^\top \mathbf{y}}{\tau})$ ，逆信噪比为

$$\text{SNR}_{\text{exp}}^{-1} = N \frac{\mathbb{E}_{\mathbf{k}_j} [\exp(\frac{2}{\tau} \mathbf{k}_j^\top \mathbf{k}_i)]}{\exp(\frac{2}{\tau} \|\mathbf{k}_i\|_2^2)}.$$

在给定探测键 $\mathbf{k}_i$ 的条件下，内积 $X_j = \mathbf{k}_j^\top \mathbf{k}_i \sim \mathcal{N}(0, \|\mathbf{k}_i\|_2^2)$ 。因此，高斯矩生成函数给出 $\mathbb{E}[\exp(\frac{2X_j}{\tau})] = \exp(\frac{2\|\mathbf{k}_i\|_2^2}{\tau^2})$ ，从而

$$\text{SNR}_{\text{exp}}^{-1} = N \exp\left(\frac{2}{\tau^2} - \frac{2}{\tau}\right) \|\mathbf{k}_i\|_2^2 = N \exp\left(\frac{2(1-\tau)}{\tau^2} \|\mathbf{k}_i\|_2^2\right).$$

将 $\mathbf{k}_i$ 的平方范数替换为 d_k 得到

$$\text{SNR}_{\text{exp}}^{-1} = \frac{N}{\exp\left(\frac{2(\tau-1)}{\tau^2}(d_k + \mathcal{O}(\sqrt{d_k}))\right)}.$$

ReLU kernel. 设 $\kappa_{\text{ReLU}}(\mathbf{x}, \mathbf{y}) = \text{ReLU}(\mathbf{x}^\top \mathbf{y}) = \max(0, \mathbf{x}^\top \mathbf{y})$ 且内积 $X_j = \mathbf{k}_j^\top \mathbf{k}_i$ 是高斯分布的， $X_j \mid \mathbf{k}_i \sim \mathcal{N}(0, \|\mathbf{k}_i\|_2^2)$ 。由于高斯密度是对称的，

$$\mathbb{E}[\text{ReLU}(X_j)^2 \mid \mathbf{k}_i] = \mathbb{E}[X_j^2 \mathbf{1}_{\{X_j > 0\}}] = \frac{1}{2} \mathbb{E}[X_j^2] = \frac{\|\mathbf{k}_i\|_2^2}{2}.$$

同时 $\kappa_{\text{ReLU}}(\mathbf{k}_i, \mathbf{k}_i) = \text{ReLU}(\|\mathbf{k}_i\|_2^2) = \|\mathbf{k}_i\|_2^2$ ，因此逆信噪比为

$$\text{SNR}_{\text{ReLU}}^{-1} = N \frac{\|\mathbf{k}_i\|_2^2/2}{\|\mathbf{k}_i\|_2^4} = \frac{N}{2 \|\mathbf{k}_i\|_2^2}.$$

将 $\mathbf{k}_i$ 的平方范数替换为 d_k 得到

$$\text{SNR}_{\text{ReLU}}^{-1} = \frac{N}{2(d_k + \mathcal{O}(\sqrt{d_k}))}.$$

SoLU kernel. 对于 Softmax-Linear-Unit (SoLU) 非线性, 我们定义核

$$\kappa_{\text{SoLU}}(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^\top \mathbf{y}) \exp\left(\frac{\mathbf{x}^\top \mathbf{y}}{\tau}\right), \quad \tau \in \mathbb{R}_{>0}.$$

令 $X_j = \mathbf{k}_j^\top \mathbf{k}_i$ 如前所述。在给定查询键 $\mathbf{k}_i$ 的条件下, 随机变量 $X_j \mid \mathbf{k}_i \sim \mathcal{N}(0, s^2)$ 满足 $s^2 \stackrel{\text{def}}{=} \|\mathbf{k}_i\|_2^2$ 。由于 X_j 是高斯分布, 我们可以使用矩生成函数 $\mathbb{E}[e^{aX_j}] = \exp(a^2 s^2 / 2)$ 及其导数:

$$\mathbb{E}[X_j^2 e^{aX_j}] = (s^2 + a^2 s^4) \exp\left(\frac{a^2 s^2}{2}\right).$$

利用 $a = 2/\tau$ (因为 κ^2 包含 $\exp(2X_j/\tau)$) 我们得到

$$\begin{aligned} \mathbb{E}[\kappa_{\text{SoLU}}(\mathbf{k}_j, \mathbf{k}_i)^2 \mid \mathbf{k}_i] &= \mathbb{E}[X_j^2 e^{2X_j/\tau}] = s^2 \left(1 + \frac{4s^2}{\tau^2}\right) \exp\left(\frac{2s^2}{\tau^2}\right), \\ \kappa_{\text{SoLU}}(\mathbf{k}_i, \mathbf{k}_i) &= s^2 \exp\left(\frac{s^2}{\tau}\right) \implies \kappa_{\text{SoLU}}(\mathbf{k}_i, \mathbf{k}_i)^2 = s^4 \exp\left(\frac{2s^2}{\tau}\right). \end{aligned}$$

将这两个部分代入 $\text{SNR}^{-1} = N \mathbb{E}[\kappa^2(\mathbf{k}_j, \mathbf{k}_i)] / \kappa^2(\mathbf{k}_i, \mathbf{k}_i)$ 得到

$$\text{SNR}_{\text{SoLU}}^{-1} = N \frac{s^2(1 + \frac{4s^2}{\tau^2}) \exp(\frac{2s^2}{\tau^2})}{s^4 \exp(\frac{2s^2}{\tau})} = N \frac{1 + \frac{4s^2}{\tau^2}}{s^2} \exp\left[\frac{2s^2}{\tau^2} - \frac{2s^2}{\tau}\right].$$

对于独立同分布的标准正态键, 其平方范数为 $\|\mathbf{k}_i\|_2^2 \sim \chi_{d_k}^2$, 其均值为 d_k 。将 s^2 替换为 d_k , 得到大宽度估计

$$\text{SNR}_{\text{SoLU}}^{-1} \approx N \frac{1 + \frac{4d_k}{\tau^2}}{d_k} \exp\left[-\frac{2d_k}{\tau^2}(\tau - 1)\right].$$

设定温度 $\tau = \sqrt{d_k}$ 可得:

$$\text{SNR}_{\text{SoLU}}^{-1} \approx N \frac{1 + \frac{4d_k}{d_k}}{d_k} \exp[-2(\sqrt{d_k} - 1)] \approx \frac{5N}{d_k \exp(2\sqrt{d_k})}.$$

B Optimization Objective of Associative Memory

B.1 General Form

考虑一种通用的记忆更新循环形式:

$$\mathbf{S}_t = \mathbf{A}_t \mathbf{S}_{t-1} \mathbf{B}_t + \mathbf{C}_t, \tag{59}$$

其中 $\mathbf{A}_t$ 、 $\mathbf{B}_t$ 和 $\mathbf{C}_t$ 是参数矩阵。

我们的目标是构建一个标量优化目标 $\mathcal{L}_t(\mathbf{S}_{t-1})$ ，使得：

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \frac{\partial \mathcal{L}_t(\mathbf{S}_{t-1})}{\partial \mathbf{S}_{t-1}}. \quad (60)$$

换言之，我们要求：

$$\frac{\partial \mathcal{L}_t(\mathbf{S}_{t-1})}{\partial \mathbf{S}_{t-1}} = \mathbf{S}_{t-1} - \mathbf{A}_t \mathbf{S}_{t-1} \mathbf{B}_t - \mathbf{C}_t. \quad (61)$$

该梯度场的线性部分可表示为：

$$\mathbf{S}_{t-1} \mapsto \mathbf{S}_{t-1} - \mathbf{A}_t \mathbf{S}_{t-1} \mathbf{B}_t. \quad (62)$$

为确保一个向量场在矩阵空间（配备 Frobenius 内积 $\langle \mathbf{X}, \mathbf{Y} \rangle = \text{tr}(\mathbf{X}^\top \mathbf{Y})$ ）中是一个有效的梯度场，它必须是自伴的。具体而言，对于任意矩阵 $\mathbf{X}, \mathbf{Y}$ ，我们需要：

$$\langle \mathbf{X}, \mathbf{A}_t \mathbf{Y} \mathbf{B}_t \rangle = \langle \mathbf{A}_t \mathbf{X} \mathbf{B}_t, \mathbf{Y} \rangle. \quad (63)$$

此条件要求 $\mathbf{A}_t$ 和 $\mathbf{B}_t$ 为对称矩阵。如果它们不对称，通常不存在一个标量势函数，其梯度能产生所需的向量场（式 62）。

假设 $\mathbf{A}_t$ 和 $\mathbf{B}_t$ 是对称的，我们可以显式地构造 $\mathcal{L}_t(\mathbf{S}_{t-1})$ 如下：

$$\mathcal{L}_t(\mathbf{S}_{t-1}) = \frac{1}{2} \text{tr}(\mathbf{S}_{t-1}^\top \mathbf{S}_{t-1}) - \frac{1}{2} \text{tr}(\mathbf{S}_{t-1}^\top \mathbf{A}_t \mathbf{S}_{t-1} \mathbf{B}_t) - \text{tr}(\mathbf{C}_t^\top \mathbf{S}_{t-1}), \quad (64)$$

其中各项的梯度为：

- 第一项的梯度：

$$\frac{\partial}{\partial \mathbf{S}_{t-1}} \frac{1}{2} \text{tr}(\mathbf{S}_{t-1}^\top \mathbf{S}_{t-1}) = \mathbf{S}_{t-1};$$

- 第二项的梯度（假设对称的 $\mathbf{A}_t$ 和 $\mathbf{B}_t$ ）：

$$\frac{\partial}{\partial \mathbf{S}_{t-1}} \frac{1}{2} \text{tr}(\mathbf{S}_{t-1}^\top \mathbf{A}_t \mathbf{S}_{t-1} \mathbf{B}_t) = \mathbf{A}_t \mathbf{S}_{t-1} \mathbf{B}_t;$$

- 第三项的梯度：

$$\frac{\partial}{\partial \mathbf{S}_{t-1}} \text{tr}(\mathbf{C}_t^\top \mathbf{S}_{t-1}) = \mathbf{C}_t.$$

B.2 Representative Cases

Linear Attention 通过与线性注意力的记忆更新公式比较：

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad (65)$$

我们得到： $\mathbf{A}_t = \mathbf{I}$ 、 $\mathbf{B}_t = \mathbf{I}$ 和 $\mathbf{C}_t = \mathbf{v}_t \mathbf{k}_t^\top$ 。将这些代入公式 64，可得：

$$\mathcal{L}_t(\mathbf{S}_{t-1}) = \frac{1}{2} \text{tr}(\mathbf{S}_{t-1}^\top \mathbf{S}_{t-1}) - \frac{1}{2} \text{tr}(\mathbf{S}_{t-1}^\top \mathbf{S}_{t-1}) - \text{tr}((\mathbf{v}_t \mathbf{k}_t^\top)^\top \mathbf{S}_{t-1}). \quad (66)$$

由于，

$$\begin{aligned} \text{tr}((\mathbf{v}_t \mathbf{k}_t^\top)^\top \mathbf{S}_{t-1}) &= \text{tr}(\mathbf{k}_t \mathbf{v}_t^\top \mathbf{S}_{t-1}) \\ &= \text{tr}(\mathbf{S}_{t-1} \mathbf{k}_t \mathbf{v}_t^\top) \\ &= \mathbf{v}_t^\top \mathbf{S}_{t-1} \mathbf{k}_t \\ &= \langle \mathbf{S}_{t-1} \mathbf{k}_t, \mathbf{v}_t \rangle, \end{aligned} \quad (67)$$

因此我们有：

$$\mathcal{L}_t(\mathbf{S}_{t-1}) = -\langle \mathbf{S}_{t-1} \mathbf{k}_t, \mathbf{v}_t \rangle. \quad (68)$$

如果我们引入门控矩阵 $\text{diag}(\boldsymbol{\lambda}_t)$ ，即 $\mathbf{A}_t = \text{diag}(\boldsymbol{\lambda}_t)$ ，那么：

$$\begin{aligned} \mathcal{L}_t(\mathbf{S}_{t-1}) &= \frac{1}{2} \text{tr}(\mathbf{S}_{t-1}^\top \mathbf{S}_{t-1}) - \frac{1}{2} \text{tr}(\mathbf{S}_{t-1}^\top \text{diag}(\boldsymbol{\lambda}_t) \mathbf{S}_{t-1}) - \text{tr}((\mathbf{v}_t \mathbf{k}_t^\top)^\top \mathbf{S}_{t-1}) \\ &= \frac{1}{2} \text{diag}(1 - \boldsymbol{\lambda}_t) \|\mathbf{S}_{t-1}\|_F^2 - \langle \mathbf{S}_{t-1} \mathbf{k}_t, \mathbf{v}_t \rangle. \end{aligned} \quad (69)$$

DeltaNet 通过与 DeltaNet 的记忆更新公式比较：

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \mathbf{k}_t \mathbf{k}_t^\top) + \mathbf{v}_t \mathbf{k}_t^\top, \quad (70)$$

我们得到： $\mathbf{A}_t = \mathbf{I}$ 、 $\mathbf{B}_t = \mathbf{I} - \mathbf{k}_t \mathbf{k}_t^\top$ 和 $\mathbf{C}_t = \mathbf{v}_t \mathbf{k}_t^\top$ 。将这些代入公式 64，可得：

$$\mathcal{L}_t(\mathbf{S}_{t-1}) = \frac{1}{2} \text{tr}(\mathbf{S}_{t-1}^\top \mathbf{S}_{t-1} \mathbf{k}_t \mathbf{k}_t^\top) - \text{tr}(\mathbf{k}_t \mathbf{v}_t^\top \mathbf{S}_{t-1}), \quad (71)$$

其中

$$\begin{aligned} \text{tr}(\mathbf{S}_{t-1}^\top \mathbf{S}_{t-1} \mathbf{k}_t \mathbf{k}_t^\top) &= \text{tr}((\mathbf{S}_{t-1} \mathbf{k}_t)(\mathbf{S}_{t-1} \mathbf{k}_t)^\top) \\ &= \|\mathbf{S}_{t-1} \mathbf{k}_t\|^2. \end{aligned} \quad (72)$$

因此，

$$\begin{aligned} \mathcal{L}_t(\mathbf{S}_{t-1}) &= \frac{1}{2} \|\mathbf{S}_{t-1} \mathbf{k}_t\|^2 - \langle \mathbf{S}_{t-1} \mathbf{k}_t, \mathbf{v}_t \rangle \\ &= \frac{1}{2} \|\mathbf{S}_{t-1} \mathbf{k}_t - \mathbf{v}_t\|^2 - \frac{1}{2} \|\mathbf{v}_t\|^2. \end{aligned} \quad (73)$$

由于 $-\frac{1}{2}\|\mathbf{v}_t\|^2$ 与 $\mathbf{S}_{t-1}$ 无关, 因此在计算梯度后消失, 优化目标可表示为:

$$\mathcal{L}_t(\mathbf{S}_{t-1}) = \frac{1}{2}\|\mathbf{S}_{t-1}\mathbf{k}_t - \mathbf{v}_t\|^2. \quad (74)$$

引入动量, 即 $\mathbf{C}_t = \eta_t \mathbf{C}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$, 其中 η 为动量系数且 $\eta \in (0, 1)$, 我们将此循环公式展开如下:

$$\mathbf{C}_t = \sum_{i=1}^t \left(\prod_{j=i+1}^t \eta_j \right) \mathbf{v}_i \mathbf{k}_i^\top. \quad (75)$$

这相当于在原始的 $\mathbf{C}_t$ 上增加了一个额外常数系数项, 将优化目标更新为:

$$\mathcal{L}_t(\mathbf{S}_{t-1}) = \frac{1}{2}\|\mathbf{S}_{t-1}\mathbf{k}_t\|^2 - \sum_{i=1}^t \left(\prod_{j=i+1}^t \eta_j \right) \langle \mathbf{S}_{t-1}\mathbf{k}_t, \mathbf{v}_i \rangle. \quad (76)$$

Softmax Attention 考虑输出 $\mathbf{o}_t$:

$$\begin{aligned} \mathbf{o}_t &= \sum_{i=1}^t \frac{\exp(\mathbf{q}_t^\top \mathbf{k}_i)}{\sum_{j=1}^t \exp(\mathbf{q}_t^\top \mathbf{k}_j)} \mathbf{v}_i \\ &= \sum_{i=1}^t \frac{\phi(\mathbf{q}_t)^\top \phi(\mathbf{k}_i)}{\sum_{j=1}^t \exp(\mathbf{q}_t^\top \mathbf{k}_j)} \mathbf{v}_i, \end{aligned} \quad (77)$$

其中 $\phi(\cdot)$ 由方程 11 定义。对于足够大的 t , 根据大数定律, 我们有 $\sum_{j=1}^t \exp(\mathbf{q}_t^\top \mathbf{k}_j) \approx tc(\mathbf{q}_t)$, 其中 $c(\mathbf{q}_t)$ 是一个查询相关的常数。因此, 输出可近似为:

$$\mathbf{o}_t \approx \frac{1}{c(\mathbf{q}_t)} \sum_{i=1}^t \frac{\phi(\mathbf{q}_t)^\top \phi(\mathbf{k}_i)}{t} \mathbf{v}_i. \quad (78)$$

令 $\mathbf{S}_t = \sum_{i=1}^t \frac{\mathbf{v}_i \phi(\mathbf{k}_i)^\top}{t}$; 则有:

$$\mathbf{S}_t = \frac{t-1}{t} \mathbf{S}_{t-1} + \frac{1}{t} \mathbf{v}_t \phi(\mathbf{k}_t)^\top. \quad (79)$$

这意味着 $\mathbf{A}_t = \frac{t-1}{t} \mathbf{I}$ 、 $\mathbf{B}_t = \mathbf{I}$ 和 $\mathbf{C}_t = \frac{1}{t} \mathbf{v}_t \phi(\mathbf{k}_t)^\top$ 。将这些代入方程 64, 我们得到:

$$\mathcal{L}_t(\mathbf{S}_{t-1}) = \frac{1}{t} \left(\frac{1}{2} \|\mathbf{S}_{t-1}\|_F^2 - \langle \mathbf{S}_{t-1} \phi(\mathbf{k}_t), \mathbf{v}_t \rangle \right). \quad (80)$$

如果我们引入门控矩阵 $\text{diag}(\boldsymbol{\lambda}_t)$, 即 $\mathbf{A}_t = \frac{t-1}{t} \text{diag}(\boldsymbol{\lambda}_t)$, 则:

$$\mathcal{L}_t(\mathbf{S}_{t-1}) = \frac{1}{t} \left(\frac{1}{2} \text{diag}(1 - \boldsymbol{\lambda}_t) \|\mathbf{S}_{t-1}\|_F^2 - \langle \mathbf{S}_{t-1} \phi(\mathbf{k}_t), \mathbf{v}_t \rangle \right). \quad (81)$$

C Delta Rule with Kernel Function

我们考虑核函数 $\kappa(\mathbf{x}, \mathbf{y}) = f(\mathbf{x})^\top f(\mathbf{y})$, 其中 $f(\mathbf{x})$ 是一个从 d 到无限维的映射。那么基于 delta 规则的更新形式以及相应的读出方程可以重写为:

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - f(\mathbf{k}_t)f(\mathbf{k}_t)^\top) + \mathbf{v}_t f(\mathbf{k}_t)^\top, \quad (82)$$

$$\mathbf{o}_t = \mathbf{S}_t f(\mathbf{q}_t). \quad (83)$$

假设:

$$\mathbf{S}_t = \sum_{i=1}^t \mathbf{u}_i \mathbf{w}_i^\top, \quad (84)$$

其中 $\mathbf{u}_i$ 和 $\mathbf{w}_i$ 待定。于是我们有:

$$\sum_{i=1}^t \mathbf{u}_i \mathbf{w}_i^\top = \sum_{i=1}^{t-1} \mathbf{u}_i \mathbf{w}_i^\top (\mathbf{I} - f(\mathbf{k}_t)f(\mathbf{k}_t)^\top) + \mathbf{v}_t f(\mathbf{k}_t)^\top, \quad (85)$$

$$\mathbf{u}_t \mathbf{w}_t^\top = \sum_{i=1}^{t-1} \mathbf{u}_i \mathbf{w}_i^\top (-f(\mathbf{k}_t)f(\mathbf{k}_t)^\top) + \mathbf{v}_t f(\mathbf{k}_t)^\top, \quad (86)$$

取待定的 $\mathbf{w}_i = f(\mathbf{k}_i)$:

$$\begin{aligned} \mathbf{u}_t f(\mathbf{k}_t)^\top &= \sum_{i=1}^{t-1} \mathbf{u}_i f(\mathbf{k}_i)^\top (-f(\mathbf{k}_t)f(\mathbf{k}_t)^\top) + \mathbf{v}_t f(\mathbf{k}_t)^\top \\ &= - \sum_{i=1}^{t-1} f(\mathbf{k}_i)^\top f(\mathbf{k}_t) \mathbf{u}_i f(\mathbf{k}_t)^\top + \mathbf{v}_t f(\mathbf{k}_t)^\top \\ &= \left(- \sum_{i=1}^{t-1} f(\mathbf{k}_i)^\top f(\mathbf{k}_t) \mathbf{u}_i + \mathbf{v}_t \right) f(\mathbf{k}_t)^\top. \end{aligned} \quad (87)$$

因此, 我们得到待定的 $\mathbf{u}_t$:

$$\begin{aligned} \cancel{\mathbf{u}_t f(\mathbf{k}_t)^\top} &= \left(- \sum_{i=1}^{t-1} f(\mathbf{k}_i)^\top f(\mathbf{k}_t) \mathbf{u}_i + \mathbf{v}_t \right) \cancel{f(\mathbf{k}_t)^\top} \\ &= - \sum_{i=1}^{t-1} \kappa(\mathbf{k}_i, \mathbf{k}_t) \mathbf{u}_i + \mathbf{v}_t. \end{aligned} \quad (88)$$

于是我们得到最终的更新形式及相应的读出方程:

$$\mathbf{S}_t = \sum_{i=1}^t \mathbf{u}_i f(\mathbf{k}_i)^\top = \mathbf{S}_{t-1} + \mathbf{u}_t \phi(\mathbf{k}_t)^\top, \quad (89)$$

$$\mathbf{o}_t = \sum_{i=1}^t \kappa(\mathbf{k}_i, \mathbf{q}_t) \mathbf{u}_i. \quad (90)$$

D Re-examining DeltaNet from the Perspective of Information Aggregation

在本节中，我们阐述了将 DeltaNet [33, 48] 与 softmax 注意力机制相结合的初步动机与推导。但这与正文的叙述逻辑有显著差异，且可能不如正文易于理解，故将其置于附录中。

D.1 Information Aggregation

我们首先考虑标准的 Transformer 注意力。为简化起见，忽略指数核函数与归一化项，其对应的联想记忆更新为：

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top. \quad (91)$$

在此形式中，存储信息 $\{\mathbf{v}_1, \mathbf{v}_2, \dots\}$ 在时间步之间相互独立，可以并行计算。现在，考虑一种新的值表示形式 $\mathbf{u}$ ，其定义为：

$$\mathbf{u}_t = \mathbf{v}_t - \sum_{i=1}^{t-1} (\mathbf{k}_i^\top \mathbf{k}_t) \mathbf{u}_i. \quad (92)$$

在这种情况下，存储信息 $\{\mathbf{u}_1, \mathbf{u}_2, \dots\}$ 不再独立，而是具有循环依赖结构。因此，联想记忆更新变为：

$$\begin{aligned} \mathbf{S}_t &= \mathbf{S}_{t-1} + \mathbf{u}_t \mathbf{k}_t^\top \\ &= \mathbf{S}_{t-1} + \left(\mathbf{v}_t - \sum_{i=1}^{t-1} (\mathbf{k}_i^\top \mathbf{k}_t) \mathbf{u}_i \right) \mathbf{k}_t^\top \\ &= \mathbf{S}_{t-1} + \left(\mathbf{v}_t - \left(\sum_{i=1}^{t-1} \mathbf{u}_i \mathbf{k}_i^\top \right) \mathbf{k}_t \right) \mathbf{k}_t^\top \\ &= \mathbf{S}_{t-1} + (\mathbf{v}_t - \mathbf{S}_{t-1} \mathbf{k}_t) \mathbf{k}_t^\top \\ &= \mathbf{S}_{t-1} (\mathbf{I} - \mathbf{k}_t \mathbf{k}_t^\top) + \mathbf{v}_t \mathbf{k}_t^\top. \end{aligned} \quad (93)$$

令人惊讶的是，我们发现这一公式恢复了 DeltaNet。我们从另一个视角推导出了 DeltaNet。公式 92 中的信息聚合方法本质上对应于 DeltaNet。

直观上，标准注意力只是将每个时间步的值写入记忆。相比之下，delta 规则（公式 92）首先基于当前值与历史值的键相似性，从当前值中移除冗余信息，仅写入增量信息。因此，后一种聚合机制同时支持记忆的写入与删除，而前者仅支持写入。

D.2 Enhancing Retrieval Capability of DeltaNet

此外，我们可以通过引入核函数来增强标准 DeltaNet 的信息检索能力。具体而言，我们将公式 92 中的键相似性计算替换为指数核函数：

$$\mathbf{u}_t = \mathbf{v}_t - \sum_{i=1}^{t-1} \frac{\exp(\mathbf{k}_i^\top \mathbf{k}_t)}{\mathbf{Z}_t^{(1)}} \mathbf{u}_i, \quad (94)$$

其中，类似于 softmax 注意力，我们引入了归一化项 $\mathbf{Z}_t^{(1)} = \sum_{i=1}^{t-1} \exp(\mathbf{k}_i^\top \mathbf{k}_t)$ 。因此，当前输出 $\mathbf{o}_t$ 计算如下：

$$\mathbf{o}_t = \sum_{i=1}^t \frac{\exp(\mathbf{k}_i^\top \mathbf{q}_t)}{\mathbf{Z}_t^{(2)}} \mathbf{u}_i, \quad (95)$$

其中 $\mathbf{Z}_t^{(2)} = \sum_{i=1}^t \exp(\mathbf{k}_i^\top \mathbf{q}_t)$ 。注意，归一化项 $\mathbf{Z}_t^{(1)}$ 和 $\mathbf{Z}_t^{(2)}$ 也可以使用 RMSNorm [49] 来稳定训练。总之，我们成功地将 delta 规则与 softmax 注意力相结合。

D.3 Efficient Chunk-wise Implementation

考虑 $\mathbf{q}, \mathbf{k}, \mathbf{v}, \mathbf{u} \in \mathbb{R}^d$ 、序列长度 T ，并假设 $T \gg d$ 。如果我们根据方程 94 计算 $\{\mathbf{u}_1, \dots, \mathbf{u}_T\}$ ，其计算复杂度为 $\mathcal{O}(T^2d)$ 。需要注意的是，由于每个 $\mathbf{u}_t$ 依赖于所有先前的 $\mathbf{u}_{<t}$ ，此算法无法并行化，导致训练时间复杂度为 $\mathcal{O}(T)$ ，显著慢于标准 softmax 注意力的 $\mathcal{O}(1)$ 复杂度。因此，它未能利用 GPU 的并行计算能力。

事实上， $\{\mathbf{u}_1, \dots, \mathbf{u}_T\}$ 的计算可以通过增加计算量来实现并行化。将方程 94 改写为矩阵形式，我们得到：

$$\mathbf{U} = \mathbf{V} - \mathbf{A}\mathbf{U}, \quad (96)$$

其中矩阵 $\mathbf{U}$ 和 $\mathbf{V}$ 的第 t 行分别是 $\mathbf{U}_{t,:} = \mathbf{u}_t$ 和 $\mathbf{V}_{t,:} = \mathbf{v}_t$ 。相似度矩阵 $\mathbf{A}$ 定义为 $\mathbf{A}_{t,i} = \frac{\exp(\mathbf{k}_i^\top \mathbf{k}_t)}{\mathbf{Z}_t^{(1)}}$ 且是下三角矩阵。因此， $\mathbf{U}$ 可以通过矩阵运算求解：

$$\mathbf{U} = (\mathbf{I} + \mathbf{A})^{-1}\mathbf{V}. \quad (97)$$

这种方法需要进行矩阵求逆操作，将计算复杂度增加到 $\mathcal{O}(T^3)$ ，但可以在 GPU 中并行执行。

直观上，我们可以寻求上述两种方法之间的折衷。通过将长度为 T 的序列划分为 N 个等长的块，每个块长度为 $C = \lfloor \frac{T}{N} \rfloor$ ，我们可以使用并行化的矩阵运算，依次计算每个块的 $\mathbf{u}$ 序列。结果，递归步骤从 $\mathcal{O}(T)$ 减少到 $\mathcal{O}(N)$ ，而计算复杂度则从 $\mathcal{O}(T^2d)$ 过渡到 $\mathcal{O}(T^2d + TCd + TC^2)$ 。本质上，这种方法以计算资源换取运行时间的减少。

以下是一个简单的 PyTorch 实现，作为伪代码。我们可以轻松修改核函数的选择或移除归一化项。

```
import torch
import torch.nn.functional as F
import math

def flash_attn(K_chunk, K_prev, V_prev):
    attn = K_chunk @ K_prev.transpose(-1, -2) / math.sqrt(K_chunk.shape[-1])
    z_intra = torch.logsumexp(attn, dim=-1)
    return torch.softmax(attn, dim=-1) @ V_prev, z_intra

def naive_implementation(k, n, d_model): """n 是之前的 v, v 实际上是新的 v。"""
    B, H, T, D = k.shape
    v = torch.zeros_like(n)
    for t in range(T):
```

```

    if t == 0:
        v[:, :, 0] = n[:, :, 0]
    else:
        scores = torch.matmul(k[:, :, :t], k[:, :, t].unsqueeze(-1)).squeeze(-1) / math.sqrt(d_model)
    )
    attn_probs = F.softmax(scores, dim=-1)
    v[:, :, t] = n[:, :, t] - torch.sum(attn_probs.unsqueeze(-1) * v[:, :, :t], dim=-2)
return vdef optimized_chunked_implementation(K, N, d_model, C):
    B, H, T, D = K.shape
    V = torch.zeros(B, H, T, D)
    chunk_nums = T // C
    mask = torch.tril(torch.ones(C, C), diagonal=-1).unsqueeze(0).unsqueeze(0).to(K.device)
    for chunk_num in range(chunk_nums):
        start = chunk_num * C
        end = (chunk_num + 1) * C
        K_chunk = K[:, :, start:end]
        N_chunk = N[:, :, start:end]
        if chunk_num > 0:
            intra_output, Z_intra = flash_attn(K_chunk, K[:, :, :start], V[:, :, :start])# $\mathcal{O}(TCD)$ 
            A = (K_chunk @ K_chunk.transpose(-2, -1)).masked_fill(mask[:, :, :C, :C] == 0, float("-inf"))
            / math.sqrt(d_model)# $\mathcal{O}(C^2D)$ 
            Z_inter = torch.logsumexp(A, dim=-1)
            P = N_chunk - intra_output * (1/(1 + torch.exp((Z_inter-Z_intra).unsqueeze(-1))))
            A = F.softmax(A, dim=-1) * (1/(1 + torch.exp((Z_intra-Z_inter).unsqueeze(-1))))
            A[:, :, 0, :] = 0
        else:
            A = (K_chunk @ K_chunk.transpose(-2, -1)).masked_fill(mask[:, :, :C, :C] == 0, float("-inf"))
            / math.sqrt(d_model)
            A = F.softmax(A, dim=-1)
            A[:, :, 0, :] = 0
            P = N_chunk
        Ti = torch.eye(C).unsqueeze(0).unsqueeze(0).unsqueeze(0).to(K.device) + A
        Ti_inverse = torch.inverse(Ti) ## 前向替换法  $\mathcal{O}(C^3)$  如果不使用softmax的归一化, 每个块
        可以并行求解。为了充分利用GPU, 我们可以使用  $\mathcal{O}(\log C)$  次迭代, 每次迭代使用  $\mathcal{O}(C^3)$ 
        次浮点运算, 例如  $\mathbf{a}_1, \mathbf{b}_1 = \mathbf{I} - \mathbf{A}, \mathbf{a}_{i+1}, \mathbf{b}_{i+1} = \mathbf{a}_i @ \mathbf{b}_i + \mathbf{b}_i @ \mathbf{b}_i$ , 返回  $\mathbf{a}_{\log C - 1}$ 
        V[:, :, start:end] = Ti_inverse @ P #  $\mathcal{O}(C^2D)$ 
    return V #  $\mathcal{O}(T/C * (TCD + C^2D)) = \mathcal{O}(T^2D + TCD + TC^2)$ 

def verify_equivalence():
    B = 2
    H = 2
    T = 1024
    D = 64
    C = 32
    K = torch.randn(B, H, T, D)
    N = torch.randn(B, H, T, D)
    naive_output = naive_implementation(K, N, D)
    optimized_output = optimized_chunked_implementation(K, N, D, C)
    equivalence = torch.allclose(naive_output, optimized_output, atol=1e-5)
    print(f"{equivalence}")

```

Listing 1 PyTorch-style pseudo-code.

D.3.1 Efficient Matrix Inversion on GPUs.

在我们的方法中，矩阵的块求逆是一个重要环节。先前的工作 [48] 采用了前向替换方法与块矩阵求逆的混合策略，但未能解决前向替换方法无法高效利用 GPU 并行能力的问题。为解决此问题，我们研究了在 GPU 上高效实现对角为 1 的下三角矩阵求逆的方法。

假设我们研究的矩阵为 $\mathbf{I} + \mathbf{A}$ ，其中 $\mathbf{A}$ 是一个对角线为 0 的严格下三角矩阵，其大小为 $C \times C$ 。在 [48] 的设置下， C 通常为 64。我们可以发现 $\mathbf{A}^C = 0$ 。

由此可得：

$$(\mathbf{I} + \mathbf{A})(\mathbf{I} - \mathbf{A}) \prod_{i=1}^{\log_2 C - 1} (\mathbf{I} + \mathbf{A}^{2^i}) = \mathbf{I} - \mathbf{A}^C = \mathbf{I}, \quad (98)$$

这意味着

$$(\mathbf{I} + \mathbf{A})^{-1} = (\mathbf{I} - \mathbf{A}) \prod_{i=1}^{\log_2 C - 1} (\mathbf{I} + \mathbf{A}^{2^i}) \quad (99)$$

随后，我们可以通过以下迭代计算 $(\mathbf{I} + \mathbf{A})^{-1}$ ：

$$\begin{aligned} [\mathbf{X}_0, \mathbf{Y}_0] &= [\mathbf{I} - \mathbf{A}, \mathbf{A}^2] \\ [\mathbf{X}_i, \mathbf{Y}_i] &= [\mathbf{X}_{i-1}, \mathbf{0}] + [\mathbf{X}_{i-1}, \mathbf{Y}_{i-1}] \mathbf{Y}_{i-1}, i = 1, \dots, \log_2 C - 1, \end{aligned} \quad (100)$$

最终， $(\mathbf{I} + \mathbf{A})^{-1} = \mathbf{X}_{\log_2 C - 1}$ 。

E State Tracking of DeltaFormer

我们首先提出并证明一个关于状态交换的定理。然后，我们讨论在某些条件下，DeltaFormer 能够表达前述定理所描述的注意力机制。最后，我们分析 DeltaFormer 在状态跟踪中的空间复杂度优势。

定理 1 (状态交换)

Assumption 1: 存在 n 个状态点位于一个 d 维单位球面上, 且任意两个不同状态点的内积绝对值小于等于 $\epsilon(d, n)$, 这意味着:

$$\exists \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \mathbb{R}^d \quad s.t. \quad \|\mathbf{x}_i\|_2 = 1 (\forall i), \quad \max_{i \neq j} |\mathbf{x}_i^\top \mathbf{x}_j| \leq \epsilon(d, n) < \frac{1}{8}.$$

Assumption 2: 存在一个函数 f 满足:

$$\forall \mathbf{x} \in \{-1, 0, 1, 2\}, \forall \tilde{\mathbf{x}} \in U(\mathbf{x}, 4\epsilon(d, n)) : \quad f(\tilde{\mathbf{x}}) = \mathbf{x}.$$

考虑将 n 个键值对初始化为 $\{(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_n, \mathbf{v}_n)\}$ 。键 $\{\mathbf{k}_1, \dots, \mathbf{k}_n\}$ 位于一个 d 维单位球面上并满足假设 1, 这意味着:

$$\forall i, j \in \{1, \dots, n\}, i \neq j : \quad \|\mathbf{k}_i\|_2 = 1, \quad |\mathbf{k}_i^\top \mathbf{k}_j| \leq \epsilon(n, d).$$

定义如下注意力机制:

$$\mathbf{u}_t = \mathbf{v}_t - \sum_{i=1}^{t-1} f(\mathbf{k}_i^\top \mathbf{k}_t) \mathbf{u}_i, \quad \mathbf{o}_t = \sum_{i=1}^t f(\mathbf{q}_t^\top \mathbf{k}_i) \mathbf{u}_i,$$

其中 $f(\cdot)$ 满足假设 2, 并且注意到 $\forall i \in \{1, \dots, n\}$, 由于 $f(\mathbf{k}_i^\top \mathbf{k}_i) = 0$, 我们有 $\mathbf{u}_i = \mathbf{v}_i$ 。

在当前步骤 $t, t > n$, 对应于 $\mathbf{k}_i$ 的值记为 $\tilde{\mathbf{v}}_i, i \in \{1, \dots, n\}$ 。注意, 经过 $t-1-n$ 次交换后, $\tilde{\mathbf{v}}_i$ 不一定等于最初分配的 $\mathbf{v}_i$ 。 $\forall 1 \leq t_2 < t_1 \leq n$, 要交换对应于 $\mathbf{k}_{t_1}$ 和 $\mathbf{k}_{t_2}$ 的存储值 $\tilde{\mathbf{v}}_{t_1}$ 和 $\tilde{\mathbf{v}}_{t_2}$, 只需构造:

$$\mathbf{k}_t = \mathbf{k}_{t_1} - \mathbf{k}_{t_2}, \quad \mathbf{v}_t = 0$$

当检索值时:

- 查询 $\mathbf{q}_t = \mathbf{k}_{t_1}$, 则 $\mathbf{o}_t = \tilde{\mathbf{v}}_{t_2}$;
- 查询 $\mathbf{q}_t = \mathbf{k}_{t_2}$, 则 $\mathbf{o}_t = \tilde{\mathbf{v}}_{t_1}$;
- 查询 $\mathbf{q}_t = \mathbf{k}_{t_3}, 1 \leq t_3 \leq n, t_3 \neq t_1, t_2$, 则 $\mathbf{o}_t = \tilde{\mathbf{v}}_{t_3}$ 。

这意味着对应于 $\mathbf{k}_{t_1}$ 和 $\mathbf{k}_{t_2}$ 的值的交换已完成。

E.1 Proof of Theorem 1

在证明定理 1 之前，我们引入一个辅助引理以方便证明。

引理 1 考虑定理 1，键的集合 $\{\mathbf{k}_i\}_{i=1}^n$ 满足假设 1，且 $\mathbf{k}_{>n}$ 是从 $\{\mathbf{k}_i\}_{i=1}^n$ 中选出的两个键的差。如果函数 $f(\cdot)$ 满足假设 2，则以下恒等式成立：

$$\forall 1 \leq j < i \leq n, \forall l \geq 1: \quad f((\mathbf{k}_i - \mathbf{k}_j)^\top \mathbf{k}_l) = f(\mathbf{k}_i^\top \mathbf{k}_l) - f(\mathbf{k}_j^\top \mathbf{k}_l).$$

E.1.1 Proof of Lemma 1

我们根据指标 l 的值区分两种独立情况：

Case 1: $1 \leq l \leq n$. 考虑以下子情况：

i. 若 $\mathbf{k}_i = \mathbf{k}_l$ ，则我们得到

$$f((\mathbf{k}_i - \mathbf{k}_j)^\top \mathbf{k}_l) = f(1 - \mathbf{k}_j^\top \mathbf{k}_l) = f(U(1, \epsilon)) = 1$$

$$f(\mathbf{k}_i^\top \mathbf{k}_l) - f(\mathbf{k}_j^\top \mathbf{k}_l) = 1 - f(U(0, \epsilon)) = 1.$$

ii. 若 $\mathbf{k}_j = \mathbf{k}_l$ ，则我们有

$$f((\mathbf{k}_i - \mathbf{k}_j)^\top \mathbf{k}_l) = f(\mathbf{k}_i^\top \mathbf{k}_l - 1) = f(U(-1, \epsilon)) = -1$$

$$f(\mathbf{k}_i^\top \mathbf{k}_l) - f(\mathbf{k}_j^\top \mathbf{k}_l) = f(U(0, \epsilon)) - 1 = -1.$$

iii. 若 $\mathbf{k}_i \neq \mathbf{k}_l$, $\mathbf{k}_j \neq \mathbf{k}_l$ ，则

$$f((\mathbf{k}_i - \mathbf{k}_j)^\top \mathbf{k}_l) = f(\mathbf{k}_i^\top \mathbf{k}_l - \mathbf{k}_j^\top \mathbf{k}_l) = f(U(0, 2\epsilon)) = 0$$

$$f(\mathbf{k}_i^\top \mathbf{k}_l) - f(\mathbf{k}_j^\top \mathbf{k}_l) = f(U(0, \epsilon)) - f(U(0, \epsilon)) = 0.$$

Case 2: $l > n$. 在此情况下，记 $\mathbf{k}_l = \mathbf{k}_{l_1} - \mathbf{k}_{l_2}$ ，其中 $1 \leq l_2 < l_1 \leq n$ 。考虑指标 i, j 与 l_1, l_2 之间相等关系的以下可能性：

i. 若 (i, j) 与 (l_1, l_2) 中无任何一对相等，则我们有

$$f((\mathbf{k}_i - \mathbf{k}_j)^\top \mathbf{k}_l) = f(U(0, 4\epsilon)) = 0$$

$$f(\mathbf{k}_i^\top \mathbf{k}_l) - f(\mathbf{k}_j^\top \mathbf{k}_l) = f(U(0, 2\epsilon)) - f(U(0, 2\epsilon)) = 0.$$

ii. 若恰好有一对相等，我们进一步分析：

1. 若 $i = l_1$, 则我们有

$$\begin{aligned} f((\mathbf{k}_i - \mathbf{k}_j)^\top \mathbf{k}_l) &= f(U(1, 3\epsilon)) = 1 \\ f(\mathbf{k}_i^\top \mathbf{k}_l) - f(\mathbf{k}_j^\top \mathbf{k}_l) &= f(U(1, \epsilon)) - f(U(0, 2\epsilon)) = 1. \end{aligned}$$

2. 若 $i = l_2$, 则我们有

$$\begin{aligned} f((\mathbf{k}_i - \mathbf{k}_j)^\top \mathbf{k}_l) &= f(U(-1, 3\epsilon)) = -1 \\ f(\mathbf{k}_i^\top \mathbf{k}_l) - f(\mathbf{k}_j^\top \mathbf{k}_l) &= f(U(-1, \epsilon)) - f(U(0, 2\epsilon)) = -1. \end{aligned}$$

3. 若 $j = l_1$, 则类似地

$$\begin{aligned} f((\mathbf{k}_i - \mathbf{k}_j)^\top \mathbf{k}_l) &= f(U(-1, 3\epsilon)) = -1 \\ f(\mathbf{k}_i^\top \mathbf{k}_l) - f(\mathbf{k}_j^\top \mathbf{k}_l) &= f(U(0, 2\epsilon)) - f(U(1, \epsilon)) = -1. \end{aligned}$$

4. 若 $j = l_2$, 则类似地

$$\begin{aligned} f((\mathbf{k}_i - \mathbf{k}_j)^\top \mathbf{k}_l) &= f(U(1, 3\epsilon)) = 1 \\ f(\mathbf{k}_i^\top \mathbf{k}_l) - f(\mathbf{k}_j^\top \mathbf{k}_l) &= f(U(0, 2\epsilon)) - f(U(-1, \epsilon)) = 1. \end{aligned}$$

iii. 若两对同时相等:

1. 若 $i = l_1, j = l_2$, 我们有

$$\begin{aligned} f((\mathbf{k}_i - \mathbf{k}_j)^\top \mathbf{k}_l) &= f(U(2, 2\epsilon)) = 2 \\ f(\mathbf{k}_i^\top \mathbf{k}_l) - f(\mathbf{k}_j^\top \mathbf{k}_l) &= f(U(1, \epsilon)) - f(U(-1, \epsilon)) = 2. \end{aligned}$$

2. 若 $i = l_2, j = l_1$, 这与排序条件 $j < i, l_2 < l_1$ 矛盾, 因此不可能发生。

综合以上所有情况, 我们完成了证明。

E.1.2 Formally Prove Theorem 1

我们使用数学归纳法来证明定理 1。

当 $t = n + 1$ 时:

$$\mathbf{k}_t = \mathbf{k}_{t_1} - \mathbf{k}_{t_2}, \quad (101)$$

$$\mathbf{u}_t = -\sum_{i=1}^{t-1} f(\mathbf{k}_i^\top \mathbf{k}_t) \mathbf{u}_i = -\mathbf{u}_{t_1} + \mathbf{u}_{t_2}. \quad (102)$$

如果我们读取 t_1 处的状态, 即 $\mathbf{q}_t = \mathbf{k}_{t_1}$,

$$\sum_{i=1}^t f(\mathbf{q}_t^\top \mathbf{k}_i) \mathbf{u}_i = \sum_{i=1}^t f(\mathbf{k}_{t_1}^\top \mathbf{k}_i) \mathbf{u}_i = \mathbf{u}_{t_1} + (-\mathbf{u}_{t_1} + \mathbf{u}_{t_2}) = \mathbf{u}_{t_2}. \quad (103)$$

如果我们读取 t_2 处的状态，即 $\mathbf{q}_t = \mathbf{k}_{t_2}$ ，

$$\sum_{i=1}^t f(\mathbf{q}_t^\top \mathbf{k}_i) \mathbf{u}_i = \sum_{i=1}^t f(\mathbf{k}_{t_2}^\top \mathbf{k}_i) \mathbf{u}_i = \mathbf{u}_{t_2} + (\mathbf{u}_{t_1} - \mathbf{u}_{t_2}) = \mathbf{u}_{t_1}. \quad (104)$$

如果我们读取其他状态，即 j 处的状态，其中 $j \neq t_1, t_2$ ，

$$\sum_{i=1}^t f(\mathbf{q}_t^\top \mathbf{k}_i) \mathbf{u}_i = \sum_{i=1}^t f(\mathbf{k}_j^\top \mathbf{k}_i) \mathbf{u}_i = \mathbf{u}_j. \quad (105)$$

总之，在第 $t = n + 1$ 步，根据我们的规则，可以追踪 t_1 和 t_2 之间交换的状态。

假设命题对 $t - 1$ 成立，我们考虑 t ($t > n + 1$) 的情况。

在第 t 步，

$$\mathbf{k}_t = \mathbf{k}_{t_1} - \mathbf{k}_{t_2}. \quad (106)$$

根据引理 1，我们有

$$\begin{aligned} \mathbf{u}_t &= - \sum_{i=1}^{t-1} f(\mathbf{k}_t^\top \mathbf{k}_i) \mathbf{u}_i \\ &= - \sum_{i=1}^{t-1} f(\mathbf{k}_{t_1}^\top \mathbf{k}_i) \mathbf{u}_i + \sum_{i=1}^{t-1} f(\mathbf{k}_{t_2}^\top \mathbf{k}_i) \mathbf{u}_i \\ &= -\tilde{\mathbf{v}}_{t_1} + \tilde{\mathbf{v}}_{t_2}. \end{aligned} \quad (107)$$

如果我们读取 t_1 处的状态，即 $\mathbf{q}_t = \mathbf{k}_{t_1}$ ，

$$\begin{aligned} \sum_{i=1}^t f(\mathbf{q}_t^\top \mathbf{k}_i) \mathbf{u}_i &= \sum_{i=1}^t f(\mathbf{k}_{t_1}^\top \mathbf{k}_i) \mathbf{u}_i \\ &= \sum_{i=1}^{t-1} f(\mathbf{k}_{t_1}^\top \mathbf{k}_i) \mathbf{u}_i + f(\mathbf{k}_{t_1}^\top \mathbf{k}_t) \mathbf{u}_t \\ &= \tilde{\mathbf{v}}_{t_1} + (-\tilde{\mathbf{v}}_{t_1} + \tilde{\mathbf{v}}_{t_2}) \\ &= \tilde{\mathbf{v}}_{t_2}. \end{aligned} \quad (108)$$

如果我们读取 t_2 处的状态，即 $\mathbf{q}_t = \mathbf{k}_{t_2}$ ，

$$\begin{aligned}
\sum_{i=1}^t f(\mathbf{q}_t^\top \mathbf{k}_i) \mathbf{u}_i &= \sum_{i=1}^t f(\mathbf{k}_{t_2}^\top \mathbf{k}_i) \mathbf{u}_i \\
&= \sum_{i=1}^{t-1} f(\mathbf{k}_{t_2}^\top \mathbf{k}_i) \mathbf{u}_i + f(\mathbf{k}_{t_2}^\top \mathbf{k}_t) \mathbf{u}_t \\
&= \tilde{\mathbf{v}}_{t_2} + (\tilde{\mathbf{v}}_{t_1} - \tilde{\mathbf{v}}_{t_2}) \\
&= \tilde{\mathbf{v}}_{t_1}.
\end{aligned} \tag{109}$$

如果我们读取其他状态，即 j 处的状态，其中 $j \neq t_1, t_2$ ，

$$\begin{aligned}
\sum_{i=1}^t f(\mathbf{q}_t^\top \mathbf{k}_i) \mathbf{u}_i &= \sum_{i=1}^t f(\mathbf{k}_j^\top \mathbf{k}_i) \mathbf{u}_i \\
&= \sum_{i=1}^{t-1} f(\mathbf{k}_j^\top \mathbf{k}_i) \mathbf{u}_i + f(\mathbf{k}_j^\top \mathbf{k}_t) \mathbf{u}_t \\
&= \tilde{\mathbf{v}}_j.
\end{aligned} \tag{110}$$

总之，在第 t 步，根据我们的规则，对应于 $\{\mathbf{k}_1, \dots, \mathbf{k}_n\}$ 的检索状态是正确的。

通过数学归纳法，无论交换步数 t 有多大，模型总能追踪 n 状态的交换。

E.2 Re-exam Assumptions

现在我们重新审视定理 1 中的两个假设。

要使假设 1 成立， d 与 n 之间必须满足何种关系？根据 [50] 中的定理 5.2.1，

For every $\alpha \in (0, 1)$ and $\varepsilon > 0$, there exists $c > 0$ such that for every d , one can find at least 2^{cd} $\mathbb{R}^d$ 中的单位向量，其两两内积均位于 $[\alpha - \varepsilon, \alpha + \varepsilon]$ 内。

设 $\alpha = 0.01$ 和 $\varepsilon = 0.1$ ，我们有 $\epsilon(d, n) \leq 0.11 < \frac{1}{8}$ 。这意味着，要使假设 1 成立，需要满足 $d = \mathcal{O}(\log n)$ 。

关于假设 2 中 $f(\cdot)$ 的选择，直观上，我们可以使用取整函数 $\text{round}(\cdot)$ ，即将输入舍入到最近的整数。在适当的舍入精度下，此函数可以满足该假设。但它面临需要精心设计的优化问题。因此，一个自然的想法是使用多个易于优化的核的组合来逼近取整核。考虑到 KU 缓存大小，自然会想到使用分组查询注意力（GQA）[1] 来实现它，这也是为什么我们将一般形式中 κ_1 的 $\mathbf{k}$ 解耦为 $\mathbf{w}$ 和 $\mathbf{k}$ 。解耦后， $\mathbf{w}$ 可以使用 GQA 来提高性能，而无需增加 KU 缓存：

$$\mathbf{u}_t = \alpha_t \mathbf{v}_t - \beta_t \sum_{i=1}^{t-1} \sum_{j=1}^G a_j \kappa_1(\mathbf{k}_i, \mathbf{w}_t^j) \mathbf{u}_i, \tag{111}$$

其中 G 是组数， $a_j \in \mathbb{R}$ 是每个查询的权重，其作用类似于标准注意力中的 $\mathbf{o} \text{ proj}$ 。例如，当 $G = 4$ 时，

此方法至少可以完全拟合 $\{-1, 0, 1, 2\}$ 的四个点。这意味着，理论上，一个具有四个共享头部的分组查询注意力可以表达一个满足假设 2 的 $f(\cdot)$ 。如果我们考虑 $w_t^j = j\mathbf{k}_t$ ，我们总可以设置 a_1, a_2, a_3, a_4 使得 $\sum_{j=1}^4 a_j \exp(j\mathbf{x}) = \mathbf{x}$ ，当 $\mathbf{x} \in \{-1, 0, 1, 2\}$ 时，那么我们可以发现我们至少可以追踪 S_d 。另一方面，这种在 $\mathbf{u}$ 上的分组查询注意力并不会提高线性的能力，因此线性的组合仍然是线性的。

接下来是一个有趣的实验来验证这一点。我们采用类似 GQA 的方法，头维度 $=3$ ，来追踪 5 个元素的交换，交换 16 次。如图8所示，随着查询数量的增加，使用非线性核的 DeltaFormer 执行 S_5 的能力越来越强。因此，在使用非线性核的情况下，类似 GQA 的方法也可以隐式地增加核的复杂度，从而增加模型的容量。

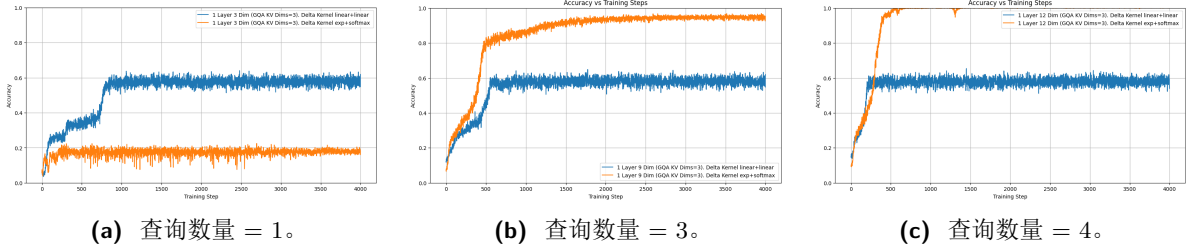


Figure 8 不同查询头数量下，具有不同 κ_1 和 κ_2 的 DeltaFormer 的比较。

总之，根据定理 1，DeltaFormer 可以实现历史时间步 t_1 和 t_2 之间的状态交换，前提是 $d = \mathcal{O}(\log n)$ 且投影矩阵 $\mathbf{W}_k$ 和 $\mathbf{W}_v$ 学习到使得 $\mathbf{k}_t = \mathbf{k}_{t_1} - \mathbf{k}_{t_2}$ 和 $\mathbf{v}_t = 0$ 的映射。

E.3 Stronger Compression Capability

如果我们想要追踪 T 次 n 状态的交换，定理 1 中的空间开销是多少？显然，我们需要设定 $d = \mathcal{O}(\log n)$ 并使用长度为 T 的 KU 缓存。因此，它需要 $\mathcal{O}(T \log n)$ 的空间。

如果我们每 $\mathcal{O}(n)$ 步基于 $\{\mathbf{k}_1, \dots, \mathbf{k}_n\}$ 读出 $\{\tilde{\mathbf{v}}_1, \dots, \tilde{\mathbf{v}}_n\}$ ，然后将 $\{(\mathbf{k}_i, \tilde{\mathbf{v}}_i)\}_{i=1}^n$ 重写入 KU 缓存。我们只使用长度为 $\mathcal{O}(n)$ 的 KU 缓存，所以总的空间开销是 $\mathcal{O}(n \log n)$ ，这远小于当 $f(\cdot)$ 是恒等映射时的 $\mathcal{O}(n^2)$ 空间，在那种情况下 Peng et al. [26] 使用一个 5×5 矩阵来追踪 5 个元素的交换。

作为比较，采用非线性核释放了 delta 规则的强大潜力：我们可以追踪指数级多状态的交换，而不仅仅是之前的 $n = \mathcal{O}(d)$ 个状态。类似地，与线性注意力相比，softmax 注意力可以从显著更长的上下文中检索更丰富的信息。

E.4 Toy Experiment

我们为 DeltaFormer 提供了一个参考代码，以完成状态跟踪的玩具实验。

```
import torch
import torch.nn as nn
import math
import torch
from torch.utils.data import Dataset, DataLoader
import random
from transformers import AdamW, get_scheduler
from tqdm import tqdm
import numpy as np
```

```

from apex.normalization import FusedRMSNorm as RMSNorm
from matplotlib import pyplot as plt
# 设置所有随机种子
def seed_all(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

seed_value = 42
seed_all(seed_value)

class RoundSTE(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x):
        return x.round()
    @staticmethod
    def backward(ctx, grad_output):
        return grad_output

class Embedding(nn.Module):
    def __init__(self, vocab_size, dim):
        super().__init__()
        self.wte = nn.Embedding(vocab_size, dim)
    def forward(self, x):
        return self.wte(x)

class Unembedding(nn.Module):
    def __init__(self, attn, dim, vocab_size):
        super().__init__()
        self.lm_head = nn.Linear(dim, vocab_size, bias=False)
    def forward(self, x):
        return self.lm_head(x)

# 支持MQA、GQA和完整MHA的广义注意力机制
class MultiHeadAttention(nn.Module):
    def __init__(self, dim, num_q_heads=8, num_kv_heads=None, rotary_base=10000):
        super().__init__()
        self.num_q_heads = num_q_heads
        self.num_kv_heads = num_kv_heads or num_q_heads # 如果为None, 则为完整MHA
        self.rotary_base = rotary_base
        self.head_dim = dim // self.num_q_heads
        self.q_proj = nn.Linear(dim, dim)
        self.k_proj = nn.Linear(dim, self.head_dim * num_kv_heads)
        self.v_proj = nn.Linear(dim, self.head_dim * num_kv_heads)
        self.out = nn.Linear(dim, dim)

    def forward(self, x):
        b, t, d = x.size()
        # 投影
        q = self.q_proj(x).view(b, t, self.num_q_heads, self.head_dim).transpose(1, 2)
        k = self.k_proj(x).view(b, t, self.num_kv_heads, self.head_dim).transpose(1, 2)
        v = self.v_proj(x).view(b, t, self.num_kv_heads, self.head_dim).transpose(1, 2)

```

```

# 计算注意力分数：如果需要，将kv头广播到q头
k_expanded = k.unsqueeze(2).expand(-1, -1, self.num_q_heads // self.num_kv_heads, -1, -1).
reshape(b, -1, t, self.head_dim)
v_expanded = v.unsqueeze(2).expand(-1, -1, self.num_q_heads // self.num_kv_heads, -1, -1).
reshape(b, -1, t, self.head_dim)
# q: b,t,Qh,hd, k_expanded: b,t,Qh,hd
scores = torch.einsum('bhtd,bhsd->bhts', q, k_expanded) / math.sqrt(d // self.num_q_heads)
attn_mask = torch.tril(torch.ones(t, t)).unsqueeze(0).unsqueeze(1).to(x.device)
scores = scores.masked_fill(attn_mask == 0, float('-inf'))
attn = torch.softmax(scores, dim=-1).to(x.dtype)
out = torch.einsum('bhts,bhsd->bthd', attn, v_expanded)
out = out.reshape(b, t, d)
return self.out(out)class DeltaAttention(MultiHeadAttention):
def __init__(self, dim, num_q_heads=8, num_kv_heads=None, rotary_base=10000, kernel_1='linear',
kernel_2='linear'):
    super().__init__(dim, num_q_heads, num_kv_heads, rotary_base)
    self.alpha = nn.Parameter(torch.ones(1), requires_grad=True)
    self.beta = nn.Parameter(torch.ones(1), requires_grad=True)
    self.kernel_1 = kernel_1
    self.kernel_2 = kernel_2
    self.w_proj = nn.Linear(dim, dim)
    self.w_weight = nn.Parameter(torch.randn(1,num_q_heads,1,1), requires_grad=True)
def forward(self, x):
    b, t, d = x.size()
    # 投影
    q = self.q_proj(x).view(b, t, self.num_q_heads, self.head_dim).transpose(1, 2)
    w = self.w_proj(x).view(b, t, self.num_q_heads, self.head_dim).transpose(1, 2)
    k = self.k_proj(x).view(b, t, self.num_kv_heads, self.head_dim).transpose(1, 2)
    v = self.v_proj(x).view(b, t, self.num_kv_heads, self.head_dim).transpose(1, 2)
    # 计算注意力分数：如果需要，将kv头广播到q头
    k_expanded = k.unsqueeze(2).expand(-1, -1, self.num_q_heads // self.num_kv_heads, -1, -1).
    reshape(b, -1, t, self.head_dim)
    # v_expanded = v.unsqueeze(2).expand(-1, -1, self.num_q_heads // self.num_kv_heads, -1, -1).
    reshape(b, -1, t, self.head_dim)

    A = torch.einsum("b h s d, b h t d-> b h s t", w, k_expanded) / math.sqrt(d // self.num_q_heads)

    attn_mask = torch.tril(torch.ones(t, t),-1).unsqueeze(0).unsqueeze(1).to(x.device)
    A = A.masked_fill(attn_mask == 0, 0)
    if self.kernel_1 == 'linear':
        A = A.float()
    elif self.kernel_1 == 'round':
        A = (RoundSTE.apply(A)).float()
    elif self.kernel_1 == 'exp':
        A = torch.exp(A).float()
    elif self.kernel_1 == 'softmax':
        A = torch.softmax(A, dim=-1).float()
    elif self.kernel_1 == 'relu':
        A = torch.relu(A).float()

    A = A*self.w_weight
    group_size = self.num_q_heads // self.num_kv_heads
    A_reshaped = A.view(b, self.num_kv_heads, group_size, t, t)
    A = A_reshaped.mean(dim=2) # 形状: (B, num_kv_heads, s, s)

```

```

        u = torch.linalg.solve_triangular(self.beta * A, self.alpha * v, upper=False, unitriangular=True)
        ).to(k_expanded)
        u = u.unsqueeze(2).expand(-1, -1, self.num_q_heads // self.num_kv_heads, -1, -1).reshape(b, -1,
        t, self.head_dim)
        scores = torch.einsum('bhtd,bhsd->bhts', q, k_expanded) / math.sqrt(d // self.num_q_heads)
        attn_mask = torch.tril(torch.ones(t, t)).unsqueeze(0).unsqueeze(1).to(x.device)

        if self.kernel_2 == 'linear':
            attn = scores.masked_fill(attn_mask == 0, 0)
            out = torch.einsum('bhts,bhsd->bthd', attn, u)
        elif self.kernel_2 == 'round':
            attn = (RoundSTE.apply(scores)).masked_fill(attn_mask == 0, 0)
            out = torch.einsum('bhts,bhsd->bthd', attn, u)
        elif self.kernel_2 == 'softmax':
            scores = scores.masked_fill(attn_mask == 0, float('-inf'))
            attn = torch.softmax(scores, dim=-1).to(x.dtype)
            out = torch.einsum('bhts,bhsd->bthd', attn, u)
        elif self.kernel_2 == 'relu':
            attn = torch.relu(scores).masked_fill(attn_mask == 0, 0)
            out = torch.einsum('bhts,bhsd->bthd', attn, u)
        out = out.reshape(b, t, d)
        return self.out(out)

class TransformerBlock(nn.Module):
    def __init__(self, dim, num_q_heads=8, num_kv_heads=None, rotary_base=10000, kernel_1=None, kernel_2
    =None):
        super().__init__()
        if kernel_1 is None:
            self.attn = MultiHeadAttention(dim, num_q_heads, num_kv_heads, rotary_base)
        else:
            self.attn = DeltaAttention(dim, num_q_heads, num_kv_heads, rotary_base, kernel_1, kernel_2)

    def forward(self, x):
        x = x + self.attn(x)
        return x

class Transformer(nn.Module):
    def __init__(self, vocab_size, dim, n_layers=1,
        num_heads=8, num_kv_heads=None, rotary_base=10000, kernel_1=None, kernel_2=None):
        super().__init__()
        self.emb = Embedding(vocab_size, dim)
        self.blocks = nn.ModuleList([
            TransformerBlock(dim, num_heads, num_kv_heads, rotary_base, kernel_1, kernel_2)
            for _ in range(n_layers)
        ])
        self.unemb = Unembedding(dim, vocab_size)

    def forward(self, x):
        x = self.emb(x)
        for block in self.blocks:
            x = block(x)
        return self.unemb(x)

n_elements = 5
swap_pairs = [(i, j) for i in range(n_elements) for j in range(i+1, n_elements)]

def apply_swap(perm, swap_idx):

```



```

    i, j = swap_pairs[swap_idx]
    perm = list(perm)
    perm[i], perm[j] = perm[j], perm[i]
    return tuple(perm)

def generate_S5_data(k, num_samples):
    data = []

    for _ in range(num_samples):
        swap_sequence = [random.randint(0, len(swap_pairs)-1) for _ in range(k)]
        current_perm = tuple(range(n_elements))
        first_elements = []

        for swap_idx in swap_sequence:
            current_perm = apply_swap(current_perm, swap_idx)
            first_elements.append(current_perm[0])

        input_ids = torch.tensor(swap_sequence, dtype=torch.long)
        labels = torch.tensor(first_elements, dtype=torch.long)
        data.append((input_ids, labels))

    return data

class S5Dataset(Dataset):
    def __init__(self, k=4, num_samples=3):
        self.k = k
        self.num_samples = num_samples
        self.data = generate_S5_data(self.k, self.num_samples)

    def __len__(self):
        return self.num_samples

    def __getitem__(self, idx):

        input_ids, labels = self.data[idx]
        if idx == 0:
            self.data = generate_S5_data(self.k, self.num_samples)
        return {
            "input_ids": input_ids,
            "labels": labels
        }

def train_model(model, dataloader, run_name="run", log_dict=None, k=16, bs = 128):
    device = "cuda" if torch.cuda.is_available() else "cpu"
    model.to(device).to(torch.bfloat16)

    num_epochs = 4000

    optimizer = AdamW(model.parameters(), lr=0.01, weight_decay=0.01)
    num_training_steps = num_epochs * len(dataloader)
    lr_scheduler = get_scheduler(
        name="linear", optimizer=optimizer, num_warmup_steps=128, num_training_steps=num_training_steps
    )

    progress_bar = tqdm(range(num_training_steps), desc=f"Training {run_name}")
    acc_log = []

```

```

loss_log = []
loss = nn.CrossEntropyLoss()
model.train()
step = 0
for epoch in range(num_epochs):
    for batch in dataloader:
        input_ids = batch["input_ids"].to(device)
        labels = batch["labels"].to(device)
        # print(input_ids, labels)
        logits = model(input_ids)
        loss = torch.nn.functional.cross_entropy(logits.view(-1, logits.shape[-1]), labels.view(-1))
        predictions = torch.argmax(logits, dim=-1)
        correct = (predictions == labels).sum().item()
        total = labels.numel()
        accuracy = correct / total

        loss.backward()
        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()

        acc_log.append(accuracy)
        loss_log.append(loss.item())

        progress_bar.set_postfix({"loss": f"{loss.item():.4f}", "acc": f"{accuracy:.4f}"})

    step += 1
    progress_bar.update(1)

    if accuracy > 0.999:
        k = k*2
        dataset = S5Dataset(k=k, num_samples=bs)
        dataloader = DataLoader(dataset, batch_size=bs, shuffle=False)
        print(f"step: {step}, length come to {k}")
    log_dict[run_name] = {"acc": acc_log, "loss": loss_log}

bs = 32*4
k = 16
dataset = S5Dataset(k=k, num_samples=bs)
dataloader = DataLoader(dataset, batch_size=bs, shuffle=False)
n_layers = 1
vocab_size = math.comb(n_elements, 2)
dim = 12
num_heads = 4
num_kv_heads = 1
model_base = Transformer(
    vocab_size=vocab_size,
    dim=dim,
    n_layers=n_layers,
    num_heads=num_heads,
    num_kv_heads=num_kv_heads,
    rotary_base=None,
)
kernel_cmb = [
    # ['linear', 'softmax'],
    # ['round', 'round'],

```

```

        ['linear', 'linear'],
        ['exp', 'softmax'],
        # ['softmax', 'softmax'],
        # ['round', 'softmax'],
    ]
    delta_models = []
    for kernels in kernel_cmb:
        delta_models.append(Transformer(
            vocab_size=vocab_size,
            dim=dim,
            n_layers=n_layers,
            num_heads=num_heads,
            num_kv_heads=num_kv_heads,
            rotary_base=None,
            kernel_1=kernels[0],
            kernel_2=kernels[1],
        ))

    log_dict = {}
    # train_model(model_base, dataloader, run_name=f'{n_layers} Layer {dim} Dim Transformer', log_dict=
    log_dict)
    for i, kernels in enumerate(kernel_cmb):
        train_model(delta_models[i], dataloader, run_name=f'{n_layers} Layer {dim} Dim (GQA KV Dims=3).
        Delta Kernel {kernels[0]}+{kernels[1]} ', log_dict=log_dict, k=k, bs =bs)

    plt.figure(figsize=(12, 6))
    for key in log_dict:
        accs = log_dict[key]['acc']
        steps = np.arange(len(accs))
        plt.plot(steps, accs, label=key)

    plt.title("准确率 vs 训练步数")
    plt.xlabel("训练步数")
    plt.ylabel("准确率")
    plt.ylim(0,1)
    plt.legend()
    plt.grid(True)
    plt.show()

```

Listing 2 Toy model code for exploration.

F Analytical Solution of Associative Memory

不失一般性，考虑以 L2 范数表示的内存优化目标：

$$\mathcal{L}_t(\mathbf{S}_{t-1}) = \frac{1}{2} \|\mathbf{S}_{t-1} \mathbf{k}_t - \mathbf{v}_t\|^2. \quad (112)$$

我们通过最小化以下目标来寻求最优内存矩阵 $\mathbf{S}^*$ ：

$$\mathbf{S}^* = \arg \min_{\mathbf{S}} \mathbb{E}_{\mathbf{x}} \left[\frac{1}{2} \|\mathbf{S} \mathbf{W}_k \mathbf{x} - \mathbf{W}_v \mathbf{x}\|^2 \right], \quad (113)$$

其中 $\mathbf{x}$ 表示单层输入, $\mathbf{k} = \mathbf{W}_k \mathbf{x}$ 、 $\mathbf{v} = \mathbf{W}_v \mathbf{x}$ 以及 $\mathbf{W}_k$ 、 $\mathbf{W}_v$ 是测试时固定的模型权重。

该目标可以重新表述为:

$$\begin{aligned}
\mathcal{L}(\mathbf{S}) &= \mathbb{E}_{\mathbf{x}} \left[\frac{1}{2} \|(\mathbf{S} \mathbf{W}_k - \mathbf{W}_v) \mathbf{x}\|^2 \right] \\
&= \mathbb{E}_{\mathbf{x}} \left[\frac{1}{2} \mathbf{x}^\top (\mathbf{S} \mathbf{W}_k - \mathbf{W}_v)^\top (\mathbf{S} \mathbf{W}_k - \mathbf{W}_v) \mathbf{x} \right] \\
&= \text{tr} \left(\frac{1}{2} (\mathbf{S} \mathbf{W}_k - \mathbf{W}_v)^\top (\mathbf{S} \mathbf{W}_k - \mathbf{W}_v) \mathbb{E}_{\mathbf{x}} [\mathbf{x} \mathbf{x}^\top] \right), \tag{114}
\end{aligned}$$

其中 $\text{tr}(\cdot)$ 表示迹算子。由于 $\mathcal{L}(\mathbf{S})$ 是一个标量, 我们有 $\mathcal{L}(\mathbf{S}) = \text{tr}(\mathcal{L}(\mathbf{S}))$ 。对 $\mathcal{L}(\mathbf{S})$ 关于 $\mathbf{S}$ 求导可得:

$$\begin{aligned}
\frac{\partial \mathcal{L}(\mathbf{S})}{\partial \mathbf{S}} &= \frac{\partial}{\partial \mathbf{S}} \text{tr} \left(\frac{1}{2} (\mathbf{S} \mathbf{W}_k - \mathbf{W}_v)^\top (\mathbf{S} \mathbf{W}_k - \mathbf{W}_v) \mathbb{E}_{\mathbf{x}} [\mathbf{x} \mathbf{x}^\top] \right) \\
&= (\mathbf{S} \mathbf{W}_k - \mathbf{W}_v) \mathbb{E}_{\mathbf{x}} [\mathbf{x} \mathbf{x}^\top] \mathbf{W}_k^\top. \tag{115}
\end{aligned}$$

令梯度为零, 我们得到最优解:

$$\mathbf{S}^* = (\mathbf{W}_v \mathbb{E}_{\mathbf{x}} [\mathbf{x} \mathbf{x}^\top] \mathbf{W}_k^\top) (\mathbf{W}_k \mathbb{E}_{\mathbf{x}} [\mathbf{x} \mathbf{x}^\top] \mathbf{W}_k^\top)^{-1}. \tag{116}$$